WEB SERVER SECURITY PROJECT

การจัดทำและยกระดับความปลอดภัยระบบเว็บเซิร์ฟเวอร์ WordPress บน VPS

รายงานผลการดำเนินงาน Stage 1 และ Stage 2

we-love-network.duckdns.org

ประเภทเอกสาร	รายงานโครงงานกลุ่ม
รายวิชา	88633065 เครือข่ายคอมพิวเตอร์
เสนอ	อาจารย์ภูสิต กุลเกษม
จัดทำโดย	คณะผู้จัดทำ 8 คน (รายชื่อในหน้าข้อมูลเอกสาร)
ภาคการศึกษา	ภาคเรียนที่ 1 ปีการศึกษา 2569
เวอร์ชัน / วันที่	1.0 · ตุลาคม 2569

ข้อมูลเอกสาร

รายการ	รายละเอียด
ชื่อเอกสาร	การจัดทำและยกระดับความปลอดภัยระบบเว็บเซิร์ฟเวอร์ WordPress บน VPS
ระบบที่อ้างถึง	https://we-love-network.duckdns.org (34.87.170.1)
รายวิชา	88633065 เครือข่ายคอมพิวเตอร์ ภาคเรียนที่ 1 ปีการศึกษา 2569
ผู้รับเอกสาร	อาจารย์ภูสิต กุลเกษม
หน่วยงาน	คณะวิทยาการสารสนเทศ มหาวิทยาลัยบูรพา

คณะผู้จัดทำ

ลำดับ	ชื่อ-นามสกุล	รหัสนิสิต
1	นายจารุกิตต์ แม้นชื่น	67160061
2	นางสาวนิรภัฏ บุตรเกสร	67160070
3	นางสาวกัญญาณัฐ น้อยเจริญ	67160085
4	นางสาวจรณพร พิมวงศ์	67160087
5	นายปัญญา	67160098
6	นายชาญนริศ แสงคำวัง	67160256
7	นายวงศกร เพิ่มพูล	67160268
8	นายณัฐชยนันท์ ยืนยง	67160419

ประวัติการแก้ไขเอกสาร

เวอร์ชัน	วันที่	รายละเอียด
1.0	ตุลาคม 2569	ฉบับส่งอาจารย์: Stage 1 (ด่านที่ 1–3) และ Stage 2

คำนำ

รายงานฉบับนี้จัดทำขึ้นเพื่อเป็นส่วนหนึ่งของรายวิชา 88633065 เครือข่ายคอมพิวเตอร์ โดยบันทึกการติดตั้งและยกระดับความปลอดภัยของระบบเว็บเซิร์ฟเวอร์ WordPress บนเครื่องเสมือนส่วนตัว (Virtual Private Server: VPS) ของ Google Cloud Platform ตามโจทย์ที่ได้รับมอบหมาย ซึ่งแบ่งออกเป็น Stage 1 จำนวน 3 ด่าน และ Stage 2 ที่มุ่งปรับสถาปัตยกรรมให้ปลอดภัยและมีประสิทธิภาพสูงขึ้น

เนื้อหาครอบคลุมการกำหนดชื่อโดเมนผ่าน DuckDNS การเปิดใช้ HTTPS ด้วยใบรับรองจาก Let's Encrypt การเข้าถึงเซิร์ฟเวอร์ด้วย SSH แบบกุญแจสาธารณะ (PKI) เท่านั้น การจำกัดการเข้าถึงเว็บไซต์เฉพาะ IP ในประเทศไทยด้วย ipset และ iptables การพัฒนาเว็บไซต์ประชาสัมพันธ์หลักสูตรวิทยาการคอมพิวเตอร์ รวมถึงการทำ Reverse Proxy การใช้ Cloudflare Worker เป็นแคชสำหรับไฟล์ static และการสแกน Log ตามรอบเวลาเพื่อแสดงผลบน Dashboard

ทุกหัวข้อแนบหลักฐานจากระบบจริง ได้แก่ ผลคำสั่งบนเซิร์ฟเวอร์ ไฟล์ตั้งค่า และผลการทดสอบ เพื่อให้ผู้อ่านตรวจสอบได้ ข้อมูลลับ เช่น รหัสผ่าน กุญแจส่วนตัว และรหัสลับที่ใช้ระหว่าง Cloudflare Worker กับเซิร์ฟเวอร์ ได้ถูกปิดหรือแทนด้วยค่าสมมติแล้ว และ IP ของผู้ใช้งานในภาพหน้าจอถูกปิดบังเพื่อความเป็นส่วนตัว

คณะผู้จัดทำขอขอบพระคุณอาจารย์ภูสิต กุลเกษม ที่ให้ความรู้และคำแนะนำตลอดการทำโครงงาน

หวังว่ารายงานฉบับนี้จะเป็นประโยชน์ต่อผู้ที่สนใจการติดตั้งและดูแลความปลอดภัยของเว็บเซิร์ฟเวอร์ หากมีข้อผิดพลาดประการใด คณะผู้จัดทำขออภัยมา ณ ที่นี้

คณะผู้จัดทำ

ตุลาคม 2569

สารบัญ

บทที่ 1 บทนำ 1

1.1 ที่มาและความสำคัญ 1

1.2 วัตถุประสงค์ 1

1.3 ขอบเขตของโครงงาน 1

1.4 ซอฟต์แวร์และเวอร์ชันที่ใช้ 2

1.5 สถาปัตยกรรมระบบ (Stage 1) 3

1.6 หน้าที่ของสมาชิก 4

บทที่ 2 ด่านที่ 1: VPS, DuckDNS, SSH และ HTTPS 5

2.1 การเตรียม VPS บน Google Cloud Platform 5

2.2 การกำหนดชื่อโดเมนด้วย DuckDNS 5

2.3 การรักษาความปลอดภัย SSH (SSH Hardening และ PKI) 5

2.4 การติดตั้ง Web Stack 6

2.5 การเปิดใช้ HTTPS ด้วย Let's Encrypt 7

2.6 สรุปผลด่านที่ 1 8

บทที่ 3 ด่านที่ 2: Firewall จำกัดการเข้าถึงเฉพาะประเทศไทย 9

3.1 แนวคิดไฟร์วอลล์สองชั้นและการลดพื้นที่การโจมตี 9

3.2 การเตรียมรายการ IP ประเทศไทย 9

3.3 กฎ iptables 10

3.4 การบันทึกกฎให้คงอยู่หลังรีบูต 11

3.5 ผลการทดสอบ 11

บทที่ 4 ด่านที่ 3: เว็บไซต์ WordPress ประชาสัมพันธ์หลักสูตร 13

4.1 การออกแบบเว็บไซต์ 13

4.2 โครงสร้างเนื้อหา 13

4.3 ภาพหน้าเว็บไซต์ 14

4.4 ปลั๊กอินและแนวทางความปลอดภัยของ WordPress 15

บทที่ 5 Stage 2: ยกระดับความปลอดภัยและประสิทธิภาพ 16

5.1 ภาพรวมสถาปัตยกรรม Stage 2 16

5.2 Reverse Proxy สำหรับ Dashboard 17

5.3 Cloudflare Worker Cache สำหรับไฟล์ static 18

5.4 ระบบสแกน Log และ Dashboard 20

บทที่ 6 ปัญหา อุปสรรค และแนวทางแก้ไข 26

บทที่ 7 สรุปผลและข้อเสนอแนะ 27

7.1 สรุปผลการดำเนินงาน 27

7.2 ข้อเสนอแนะ 27

เอกสารอ้างอิง 28

ภาคผนวก ก คำสั่งตรวจสอบระบบ 29

ภาคผนวก ข การเพิ่มผู้ดูแลระบบรายใหม่ผ่าน SSH 30

ภาคผนวก ค โค้ด Cloudflare Worker ฉบับเต็ม 31

สารบัญตาราง

ตารางที่ 1.1 ข้อกำหนดของโจทย์และสิ่งที่ดำเนินการ 1
ตารางที่ 1.2 ซอฟต์แวร์และเวอร์ชันของระบบ 2
ตารางที่ 1.3 หน้าที่ความรับผิดชอบของสมาชิก 4
ตารางที่ 2.1 ผลการตรวจสอบด่านที่ 1 8
ตารางที่ 3.1 พอร์ตและนโยบายการเข้าถึง 9
ตารางที่ 3.2 ความหมายของข้อมูลใน ipset 10
ตารางที่ 3.3 สรุปผลการทดสอบไฟร์วอลล์ 12
ตารางที่ 4.1 โครงสร้างหน้าเว็บไซต์ 13
ตารางที่ 5.1 กฎการตรวจจับใน scan_logs.py 21
ตารางที่ 5.2 คุณลักษณะที่ใช้ฝึกโมเดล Isolation Forest 22
ตารางที่ 5.3 สรุปเหตุการณ์ที่ถูกตั้งธง ณ วันที่ 29 ตุลาคม 2569 18:30 น. 24
ตารางที่ 5.4 IP ที่โมเดล Isolation Forest จัดว่าผิดปกติ 25
ตารางที่ 6.1 ปัญหาที่พบและวิธีแก้ไข 26
ตารางที่ 7.1 สรุปสถานะตามข้อกำหนดของโจทย์ 27

สารบัญภาพ

ภาพที่ 1.1 ผลคำสั่งตรวจสอบเวอร์ชันระบบปฏิบัติการ Nginx PHP และ MySQL บนเซิร์ฟเวอร์จริง 3
ภาพที่ 1.2 แผนภาพสถาปัตยกรรมระบบ Stage 1 พร้อมเวอร์ชันซอฟต์แวร์ 3
ภาพที่ 2.1 ค่าที่ OpenSSH ใช้งานจริงจากคำสั่ง sshd -T: ปิด root ปิดรหัสผ่าน และเปิดการยืนยันด้วยกุญแจ 6
ภาพที่ 2.2 การเชื่อมต่อ SSH ด้วยบัญชี root ถูกปฏิเสธ (Permission denied (publickey)) 6
ภาพที่ 2.3 ผลคำสั่ง certbot certificates แสดงใบรับรองชนิด ECDSA ของโดเมน 7
ภาพที่ 2.4 รายละเอียดใบรับรองบนเบราว์เซอร์: ออกให้ we-love-network.duckdns.org โดย Let's Encrypt 8
ภาพที่ 3.1 ผลคำสั่ง ipset list thailand_ipv4 แสดงรายการ IP ประเทศไทย 1,196 ช่วง 10
ภาพที่ 3.2 ผลคำสั่ง iptables -S แสดงนโยบายเริ่มต้นและกฎกรองตามประเทศ 10
ภาพที่ 3.3 ผลทดสอบจาก check-host.net (ส่วนที่ 1): จุดตรวจในต่างประเทศได้ผล Connection timed out 11
ภาพที่ 3.4 ผลทดสอบจาก check-host.net (ส่วนที่ 2): จุดตรวจในยุโรป อเมริกา และเอเชียได้ผล Connection timed out 12
ภาพที่ 4.1 หน้าแรกของเว็บไซต์บนคอมพิวเตอร์ ผ่าน https://we-love-network.duckdns.org 14
ภาพที่ 4.2 หน้า "เรียนอะไรบ้าง" แสดงกลุ่มวิชาหลัก 4 กลุ่มของหลักสูตร 14
ภาพที่ 4.3 หน้าแรกบนสมาร์ตโฟน เมนูเปลี่ยนเป็นปุ่มแฮมเบอร์เกอร์ตามขนาดหน้าจอ 15
ภาพที่ 5.1 แผนภาพสถาปัตยกรรม Stage 2: Cloudflare Worker, Reverse Proxy และระบบสแกน Log 16
ภาพที่ 5.2 ค่าตั้ง Nginx ที่เกี่ยวข้อง: server_name, auth_basic, proxy_pass และ listen 17
ภาพที่ 5.3 Must-Use Plugin ที่กลุ่มพัฒนา แสดงในหน้า Plugins ของ WordPress 18
ภาพที่ 5.4 ไฟล์ปลั๊กอินในโฟลเดอร์ mu-plugins เป็นของ root และแก้ไขโดย WordPress ไม่ได้ 18
ภาพที่ 5.5 สถิติภาพรวมจาก GoAccess Dashboard (รายงาน ณ วันที่ 26 ตุลาคม 2569) 20
ภาพที่ 5.6 สถานะของ log-report.service หลังทำงานรอบล่าสุด: จบการทำงานด้วย status=0/SUCCESS 21
ภาพที่ 5.7 Log Security Dashboard: ตัวเลขสรุปและ IP ที่มีคำขอมากที่สุด (ปิดบัง IP ของผู้ใช้) 23
ภาพที่ 5.8 เหตุการณ์ที่ถูกตั้งธงจากกฎตรวจจับและจากโมเดล Isolation Forest (ปิดบัง IP ของผู้ใช้) 24

บทที่ 1
บทนำ

1.1 ที่มาและความสำคัญ

เว็บไซต์ประชาสัมพันธ์หลักสูตรเป็นช่องทางหลักที่นักเรียน ผู้ปกครอง และผู้สนใจใช้ค้นหาข้อมูลหลักสูตร แผนการเรียน และกำหนดการรับสมัคร เว็บไซต์ที่เปิดให้บริการบนอินเทอร์เน็ตจึงต้องพร้อมใช้งาน ปลอดภัย และเข้าถึงได้สะดวก ขณะเดียวกันเซิร์ฟเวอร์ที่เปิดสู่สาธารณะจะถูกสแกนหาช่องโหว่และถูกพยายามเข้าสู่ระบบอยู่ตลอดเวลา ผู้ดูแลระบบจึงต้องเข้าใจทั้งการติดตั้งบริการและการลดช่องทางการโจมตีในหลายชั้นของเครือข่าย

โครงงานนี้ติดตั้ง WordPress บน VPS ของ Google Cloud Platform (GCP) ให้บริการผ่านโดเมน `we-love-network.duckdns.org` โดย Stage 1 เน้นการติดตั้งและความปลอดภัยพื้นฐานตามด่านที่ 1–3 ส่วน Stage 2 ปรับสถาปัตยกรรมเพิ่มเติมด้วย Reverse Proxy, Cloudflare Worker Cache และระบบสแกน Log พร้อม Dashboard

1.2 วัตถุประสงค์

1. เพื่อติดตั้งและให้บริการเว็บไซต์ WordPress ประชาสัมพันธ์หลักสูตรวิทยาการคอมพิวเตอร์บน VPS
2. เพื่อกำหนดชื่อโดเมนผ่าน Dynamic DNS และให้บริการผ่าน HTTPS ด้วยใบรับรอง SSL/TLS
3. เพื่อรักษาความปลอดภัยการเข้าถึงเซิร์ฟเวอร์ โดยปิด root login ปิดการยืนยันตัวตนด้วยรหัสผ่าน และใช้ SSH Key (PKI) เท่านั้น
4. เพื่อจำกัดการเข้าถึงเว็บไซต์ให้รับการเชื่อมต่อเฉพาะจาก IP ในประเทศไทย
5. เพื่อปรับสถาปัตยกรรมให้ปลอดภัยและเร็วขึ้นด้วย Reverse Proxy และแคช และสร้างระบบสแกน Log ตามรอบเวลาเพื่อตรวจหาการโจมตี

1.3 ขอบเขตของโครงงาน

ขอบเขตงานกำหนดตามโจทย์ที่อาจารย์มอบหมาย ตารางที่ 1.1
สรุปข้อกำหนดแต่ละด่านเทียบกับสิ่งที่กลุ่มดำเนินการและบทที่อธิบายรายละเอียด

ตารางที่ 1.1 ข้อกำหนดของโจทย์และสิ่งที่ดำเนินการ

ด่าน / ข้อกำหนด	สิ่งที่ดำเนินการ	บท
ด่านที่ 1: เข้าเว็บด้วย https ผ่าน DuckDNS ไปยัง WordPress บน VPS	โดเมน we-love-network.duckdns.org + ใบรับรอง Let's Encrypt	2
ด่านที่ 1 (checkpoint): SSH ปิด root, ปิด password, ใช้ PKI เท่านั้น	PermitRootLogin no, PasswordAuthentication no, กุญแจ Ed25519	2

ด่าน / ข้อกำหนด	สิ่งที่ดำเนินการ	บท
ด่านที่ 1 (checkpoint): แสดงสถาปัตยกรรมพร้อมเวอร์ชันโปรแกรม	แผนภาพสถาปัตยกรรมและตารางเวอร์ชันจากเครื่องจริง	1
ด่านที่ 2: firewall ไม่ให้เชื่อมต่อจากต่างประเทศ	ipset thailand_ipv4 (1,196 CIDR) + iptables DROP พอร์ต 80/443	3
ด่านที่ 3: WordPress ประชาสัมพันธ์หลักสูตร ใช้ theme/plugin ให้สวยงาม	ธีม Twenty Twenty-Five ปรับแต่ง 5 หน้า รองรับมือถือ	4
Stage 2: Reverse Proxy	Nginx ทำ Reverse Proxy ไปยังบริการ Dashboard ภายในเครื่อง	5
Stage 2: Cloudflare Cache	Cloudflare Worker แคชไฟล์ static พร้อมรหัสลับยืนยันต้นทาง	5
Stage 2: ใช้ AI สแกน Log ตามรอบเวลา สร้าง Dashboard	สแกนทุก 5 นาที ด้วยกฎตรวจจับ + โมเดล Machine Learning (Isolation Forest) แสดงผลบน Dashboard	5

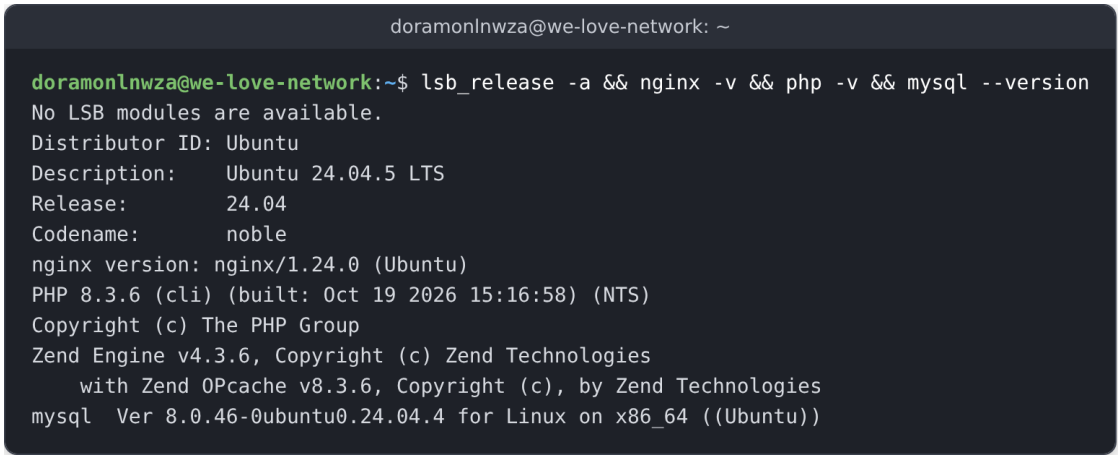
1.4 ซอฟต์แวร์และเวอร์ชันที่ใช้

เวอร์ชันในตารางที่ 1.2 ได้จากการรันคำสั่งบนเซิร์ฟเวอร์จริง (ภาพที่ 1.1) และจากหน้าผู้ดูแลระบบของ WordPress

ตารางที่ 1.2 ซอฟต์แวร์และเวอร์ชันของระบบ

องค์ประกอบ	เวอร์ชัน / รายละเอียด	ตรวจสอบด้วย
ผู้ให้บริการ VPS	Google Cloud Platform (Compute Engine) IP สาธารณะ 34.87.170.1	GCP Console
ระบบปฏิบัติการ	Ubuntu 24.04.5 LTS (noble)	`lsb_release -a`
Web Server	nginx/1.24.0 (Ubuntu)	`nginx -v`
PHP	PHP 8.3.6 (PHP-FPM) + Zend OPcache v8.3.6	`php -v`
ฐานข้อมูล	MySQL 8.0.46-0ubuntu0.24.04.4	`mysql --version`
CMS	WordPress 7.1.3	หน้า Dashboard / meta generator
ธีม	Twenty Twenty-Five	Appearance ⟶ Themes
ใบรับรอง TLS	Let's Encrypt (YE1) ชนิดกุญแจ ECDSA	`certbot certificates`
Dynamic DNS	DuckDNS: we-love-network.duckdns.org	duckdns.org
Firewall	GCP VPC Firewall + iptables + ipset	`iptables -S`
CDN / Cache	Cloudflare Workers	Cloudflare Dashboard

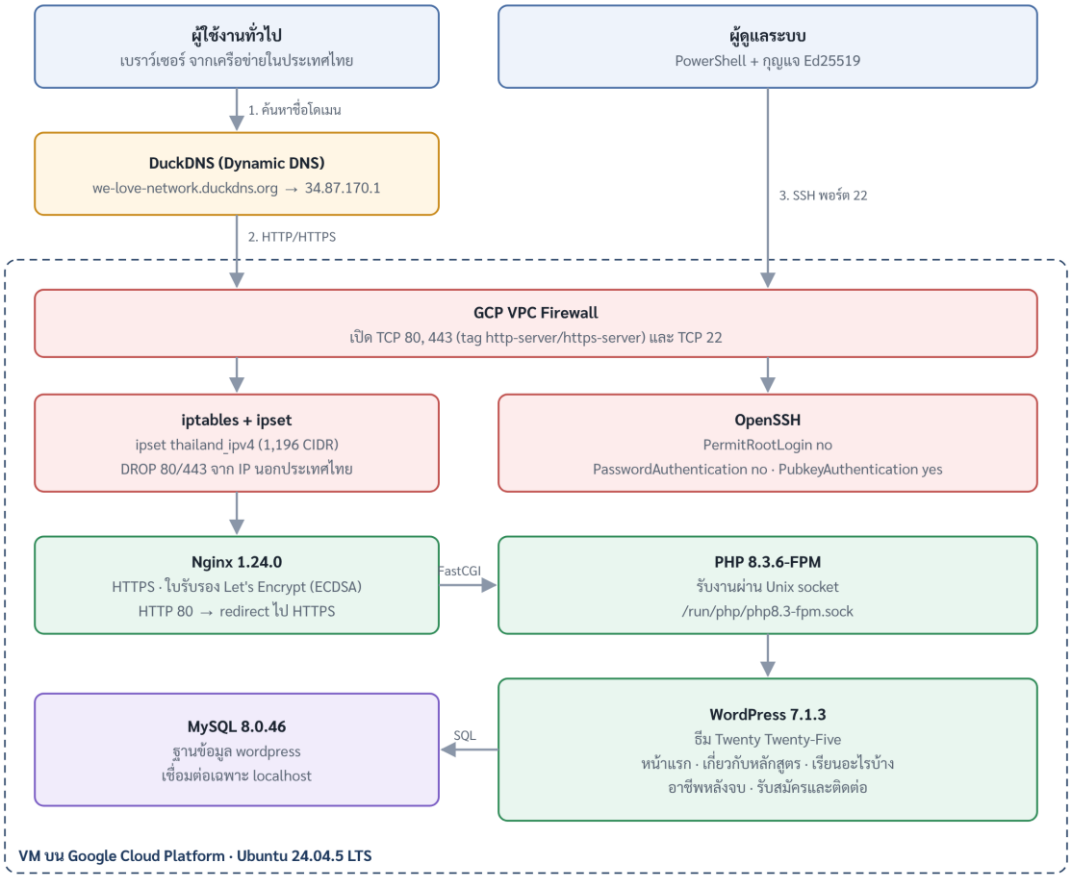
องค์ประกอบ	เวอร์ชัน / รายละเอียด	ตรวจสอบด้วย
Log Analyzer	GoAccess v1.8.1 และสคริปต์ Python 3 (scan_logs.py)	หน้า Dashboard
Machine Learning	scikit-learn (IsolationForest) ใน Python virtual environment	ai_scan_report.json



ภาพที่ 1.1 ผลคำสั่งตรวจสอบเวอร์ชันระบบปฏิบัติการ Nginx PHP และ MySQL บนเซิร์ฟเวอร์จริง

1.5 สถาปัตยกรรมระบบ (Stage 1)

ภาพที่ 1.2 แสดงเส้นทางคำขอใน Stage 1 ผู้ใช้ค้นหาชื่อโดเมนผ่าน DuckDNS แล้วเชื่อมต่อมายัง VM คำขอต้องผ่านไฟร์วอลล์สองชั้น คือ GCP VPC Firewall ที่ขอบเครือข่ายของ Google และ iptables ภายในเครื่อง จากนั้น Nginx รับคำขอ HTTPS ส่งไฟล์ PHP ไปประมวลผลที่ PHP-FPM ผ่าน Unix socket และ WordPress อ่านเขียนข้อมูลกับ MySQL ภายในเครื่อง ส่วนผู้ดูแลระบบเข้าถึงเครื่องผ่าน SSH พอร์ต 22 ด้วยกุญแจเท่านั้น



ภาพที่ 1.2 แผนภาพสถาปัตยกรรมระบบ Stage 1 พร้อมเวอร์ชันซอฟต์แวร์

1.6 หน้าที่ของสมาชิก

ตารางที่ 1.3 หน้าที่ความรับผิดชอบของสมาชิก

ชื่อ-นามสกุล	รหัสนิสิต	หน้าที่
นายจารุกิตต์ แม้นชื่น	67160061	จัดทำสไลด์นำเสนอ
นางสาวนิรภัฏ บุตรเกสร	67160070	ออกแบบเว็บไซต์ WordPress, จัดทำรายงาน
นางสาวกัญญาณัฐ น้อยเจริญ	67160085	ติดตั้งและพัฒนาเว็บไซต์ WordPress, สร้างฐานข้อมูล MySQL, จัดทำรายงาน
นางสาวจรณพร พิมพ์วงศ์	67160087	จัดทำสไลด์นำเสนอ
นายปัญญา	67160098	เช่าและติดตั้ง VPS, ตั้งค่าโดเมน DuckDNS และ HTTPS, ตั้งค่าการล็อกอิน SSH
นายชาญนริศ แสงคำวัง	67160256	ตั้งค่า Firewall จำกัดการเข้าถึงเฉพาะประเทศไทย, Cloudflare Worker Cache
นายวงศกร เพิ่มพูล	67160268	Reverse Proxy, Log Dashboard
นายณัฐชยนันท์ ยืนยง	67160419	จัดทำสไลด์นำเสนอ

บทที่ 2

ด่านที่ 1: VPS, DuckDNS, SSH และ HTTPS

โจทย์ด่านที่ 1

ต้องต่อด้วย https://abc.duckdns.org แล้วขึ้นมาที่ VPS ที่เราลง WordPress ไว้ที่นั่น

- **Checkpoint:** การต่อเข้า VPS ต่อด้วย SSH ที่ปิด root ปิด password authentication เปิด PKI เท่านั้น
- แสดงโครงสร้างของสถาปัตยกรรมที่ทำพร้อมเวอร์ชันของโปรแกรมแต่ละตัว เช่น nginx, PHP, WordPress และ MySQL

2.1 การเตรียม VPS บน Google Cloud Platform

กลุ่มสร้าง VM บน Google Cloud Platform ติดตั้ง Ubuntu Server 24.04 LTS และได้ IP สาธารณะ `34.87.170.1` ขณะสร้าง VM ได้เลือก Allow HTTP traffic และ Allow HTTPS traffic ทำให้ VM มี Network Tag `http-server` และ `https-server` ซึ่งผูกกับกฎ VPC Firewall ที่เปิดพอร์ต 80 และ 443 ส่วนพอร์ต 22 สำหรับ SSH เปิดผ่านกฎ `default-allow-ssh` ของโปรเจกต์

2.2 การกำหนดชื่อโดเมนด้วย DuckDNS

DuckDNS เป็นบริการ Dynamic DNS ฟรีที่ให้โดเมนย่อยภายใต้ `duckdns.org` กลุ่มลงทะเบียนโดเมน `we-love-network.duckdns.org` และตั้งค่า A record ให้ชี้ไปยัง `34.87.170.1` ผู้ใช้จึงเข้าเว็บไซต์ด้วยชื่อโดเมนแทนการจำ IP และใบรับรอง HTTPS ก็ออกให้กับชื่อโดเมนนี้ได้ ปัจจุบันระบบใช้เฉพาะ IPv4 ยังไม่ได้เปิด IPv6 (ไม่มี AAAA record)

2.3 การรักษาความปลอดภัย SSH (SSH Hardening และ PKI)

2.3.1 บัญชีผู้ดูแลระบบ

สร้างบัญชี `doramonlnwza` และเพิ่มเข้ากลุ่ม `sudo` เพื่อใช้ดูแลระบบแทนการเข้าสู่ระบบด้วย root โดยตรง การแยกบัญชีทำให้ตรวจสอบย้อนหลังได้ว่าใครทำอะไร และคำสั่งที่ต้องใช้สิทธิ์สูงยังต้องยืนยันรหัสผ่านของ sudo อีกชั้นหนึ่ง

```
sudo adduser doramonlnwza
sudo usermod -aG sudo doramonlnwza
```

2.3.2 การสร้างและติดตั้งกุญแจ Ed25519

SSH แบบกุญแจสาธารณะใช้การเข้ารหัสแบบอสมมาตร (asymmetric cryptography)
ผู้ดูแลสร้างกุญแจคู่บนเครื่องของตนเอง เก็บ Private Key ไว้กับตัว และนำเฉพาะ Public Key ไปใส่ในไฟล์

`authorized_keys` บนเซิร์ฟเวอร์ เมื่อเชื่อมต่อ เซิร์ฟเวอร์จะให้ไคลเอนต์พิสูจน์ว่าถือ Private Key ที่คู่กับ Public Key นั้นโดยไม่ต้องส่ง Private Key ข้ามเครือข่าย ต่างจาก HTTPS ตรงที่ SSH แบบนี้ไม่ต้องพึ่ง Certificate Authority

```
# บนเครื่องผู้ดูแล (PowerShell)
ssh-keygen -t ed25519


# บน VPS: จำกัดสิทธิ์ไฟล์ให้เจ้าของเท่านั้น
chmod 700 /home/doramonlnwza/.ssh
chmod 600 /home/doramonlnwza/.ssh/authorized_keys
```

2.3.3 การปิด Root Login และ Password Authentication

แก้ไข `/etc/ssh/sshd_config` ตามด้านล่าง ตรวจไวยากรณ์ด้วย `sshd -t` แล้ว reload บริการ ระหว่างนั้นเปิด session เดิมค้างไว้จนทดสอบ session ใหม่สำเร็จ เพื่อป้องกันการล็อกตัวเองออกจากเซิร์ฟเวอร์

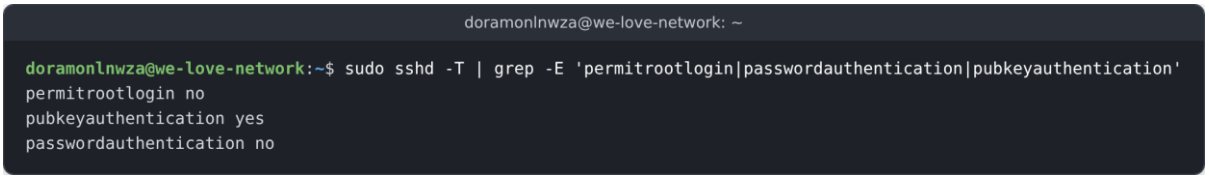
```
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes


sudo sshd -t
sudo systemctl reload ssh
```

ข้อควรระวัง: การล็อกตัวเองออกจากเซิร์ฟเวอร์

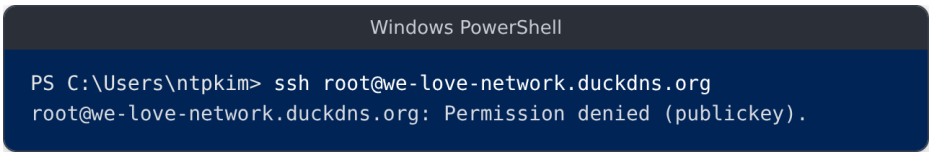
ก่อนปิด PasswordAuthentication ต้องทดสอบว่าเข้าด้วยกุญแจได้จริงใน session ใหม่ และเปิด session เดิมค้างไว้จนทดสอบเสร็จ หากกุญแจตั้งค่าผิดแล้วปิด session เดิมไป จะเข้าเซิร์ฟเวอร์ไม่ได้อีกและต้องกู้คืนผ่าน Serial Console ของ GCP

บน Ubuntu ไฟล์ในโฟลเดอร์ `/etc/ssh/sshd_config.d/` อาจทับค่าในไฟล์หลักได้
กลุ่มจึงตรวจค่าที่ระบบใช้งานจริงด้วย `sshd -T` ผลในภาพที่ 2.1 ยืนยันว่าค่าทั้งสามตรงตามโจทย์



ภาพที่ 2.1 ค่าที่ OpenSSH ใช้งานจริงจากคำสั่ง sshd -T: ปิด root ปิดรหัสผ่าน และเปิดการยืนยันด้วยกุญแจ

เมื่อทดสอบเชื่อมต่อด้วยบัญชี root เซิร์ฟเวอร์ปฏิเสธทันทีด้วยข้อความ `Permission denied (publickey)` (ภาพที่ 2.2) ข้อความนี้แสดงว่าเซิร์ฟเวอร์เสนอเฉพาะวิธี publickey ไม่มีช่องให้กรอกรหัสผ่าน จึงยืนยันได้ทั้งการปิด root และการปิด password ในคราวเดียว ส่วนบัญชี `doramonlnwza` ที่มี Private Key เข้าสู่ระบบได้ตามปกติ ดังพรอมต์ในภาพที่ 1.1 และ 2.1



ภาพที่ 2.2 การเชื่อมต่อ SSH ด้วยบัญชี root ถูกปฏิเสธ (Permission denied (publickey))

2.4 การติดตั้ง Web Stack

ติดตั้ง Nginx, PHP 8.3-FPM และ MySQL Server จากคลังแพ็กเกจของ Ubuntu Nginx รับคำขอจากผู้ใช้และส่งไฟล์ PHP ไปประมวลผลที่ PHP-FPM ผ่าน Unix socket `/run/php/php8.3-fpm.sock`
จึงไม่ต้องเปิดพอร์ตเครือข่ายเพิ่มสำหรับ PHP-FPM ส่วนค่าตั้งของ server block มีดังนี้

```
server {
    server_name we-love-network.duckdns.org;
    root /var/www/wordpress;
    index index.php;
    client_max_body_size 64M;


    location / {
        try_files $uri $uri/ /index.php?$args;
    }
    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.3-fpm.sock;
    }
    location ~ /\.ht { deny all; }


    listen 443 ssl;  # managed by Certbot
}
```

สำหรับฐานข้อมูล สร้างฐานข้อมูล `wordpress` และบัญชี `wpuser` ที่มีสิทธิ์เฉพาะฐานข้อมูลนี้และเชื่อมต่อได้จาก localhost เท่านั้นตามหลักสิทธิ์เท่าที่จำเป็น (least privilege) และเปลี่ยน Table Prefix จากค่าเริ่มต้น `wp_` เป็น `wln_` เพื่อลดผลของการโจมตี SQL injection แบบอัตโนมัติที่มักเดาชื่อตารางมาตรฐาน

```
CREATE DATABASE wordpress;
CREATE USER 'wpuser'@'localhost' IDENTIFIED BY '<รหัสผ่าน>';
GRANT ALL PRIVILEGES ON wordpress.* TO 'wpuser'@'localhost';
FLUSH PRIVILEGES;
```

2.5 การเปิดใช้ HTTPS ด้วย Let's Encrypt

ออกใบรับรองด้วย Certbot ผ่านปลั๊กอิน Nginx ซึ่งยืนยันความเป็นเจ้าของโดเมนแบบ HTTP-01 ผ่านพอร์ต 80 แล้วแก้ไข server block ให้ฟังพอร์ต 443 พร้อมตั้ง redirect จาก HTTP ไป HTTPS ให้อัตโนมัติ การต่ออายุจัดการโดย `certbot.timer` ของ systemd ซึ่งรันวันละ 2 ครั้ง

```
sudo certbot --nginx -d we-love-network.duckdns.org
```

```
doramonlnwza@we-love-network: ~
doramonlnwza@we-love-network:~$ sudo certbot certificates
Saving debug log to /var/log/letsencrypt/letsencrypt.log

- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
Found the following certs:
  Certificate Name: we-love-network.duckdns.org
    Serial Number: 6866023efc4d9bdc46504d93de5dc2e7e5e
    Key Type: ECDSA
    Domains: we-love-network.duckdns.org
    Expiry Date: 2027-01-21 09:37:22+00:00 (VALID: 84 days)
    Certificate Path: /etc/letsencrypt/live/we-love-network.duckdns.org/fullchain.pem
    Private Key Path: /etc/letsencrypt/live/we-love-network.duckdns.org/privkey.pem
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
```

ภาพที่ 2.3 ผลคำสั่ง certbot certificates แสดงใบรับรองชนิด ECDSA ของโดเมน

ใบรับรองออกโดย Let's Encrypt (Intermediate CA ชื่อ YE1) ให้กับ `we-love-network.duckdns.org` มีผลตั้งแต่วันที่ 23 ตุลาคม 2569 ถึงวันที่ 21 มกราคม 2570 (90 วัน ซึ่งเป็นอายุมาตรฐานของ Let's Encrypt) ภาพที่ 2.4 คือรายละเอียดใบรับรองที่เบราว์เซอร์ตรวจสอบได้



ภาพที่ 2.4 รายละเอียดใบรับรองบนเบราว์เซอร์: ออกให้ we-love-network.duckdns.org โดย Let's Encrypt

> **แนวคิดสำคัญ: หลักการ PKI ใน HTTPS**
> เบราว์เซอร์ตรวจลายเซ็นของใบรับรองย้อนเป็นสายไปจนถึง Root CA ที่ระบบปฏิบัติการเชื่อถือ (chain of trust) และตรวจว่าชื่อในใบรับรองตรงกับโดเมนที่เปิด เมื่อผ่านแล้วจึงใช้ TLS
> แลกเปลี่ยนกุญแจและเข้ารหัสข้อมูลระหว่างทาง ผู้ที่ดักข้อมูลกลางทางจึงอ่านหรือแก้ไขข้อมูลไม่ได้

2.6 สรุปผลด่านที่ 1

ตารางที่ 2.1 ผลการตรวจสอบด่านที่ 1

Checkpoint	ผลการทดสอบ	หลักฐาน
เข้าเว็บด้วย https://we-love-network.duckdns.org	ผ่าน	ภาพที่ 2.4, 4.1
ปิด root login ผ่าน SSH	ผ่าน (permitrootlogin no)	ภาพที่ 2.1, 2.2
ปิด password authentication	ผ่าน (passwordauthentication no)	ภาพที่ 2.1, 2.2
ใช้ PKI (SSH Key) เท่านั้น	ผ่าน (pubkeyauthentication yes, Ed25519)	ภาพที่ 2.1
แสดงสถาปัตยกรรมและเวอร์ชันโปรแกรม	ผ่าน	ภาพที่ 1.1, 1.2 และตารางที่ 1.2

บทที่ 3

ด่านที่ 2: Firewall จำกัดการเข้าถึงเฉพาะประเทศไทย

โจทย์ด่านที่ 2

มีการติดตั้ง firewall อะไรก็ได้ เช่น iptables เพื่อ block ไม่ให้มีการต่อเข้ามาจากประเทศอื่น

- **กติกา:** เว็บที่สร้างยอมให้เชื่อมจากประเทศไทยที่ server ได้เท่านั้น

3.1 แนวคิดไฟร์วอลล์สองชั้นและการลดพื้นที่การโจมตี

ระบบใช้ไฟร์วอลล์สองชั้น ชั้นแรกคือ GCP VPC Firewall ที่ขอบเครือข่ายของ Google ทำหน้าที่เปิดเฉพาะพอร์ตที่จำเป็น ชั้นที่สองคือ iptables ภายใน VM ทำหน้าที่กรองตามประเทศต้นทาง เหตุที่ไม่ใส่รายการ IP ไทยลงใน VPC Firewall โดยตรง เพราะรายการมีถึง 1,196 ช่วง ซึ่งเกินจำนวน source range ที่กฎหนึ่งข้อของ GCP รองรับ การใช้ ipset ภายในเครื่องจึงจัดการรายการขนาดใหญ่ได้ดีกว่า หลักการลดพื้นที่การโจมตี (Minimum Attack Surface) คือเปิดเฉพาะบริการที่ใช้งานจริง ดังตารางที่ 3.1

ตารางที่ 3.1 พอร์ตและนโยบายการเข้าถึง

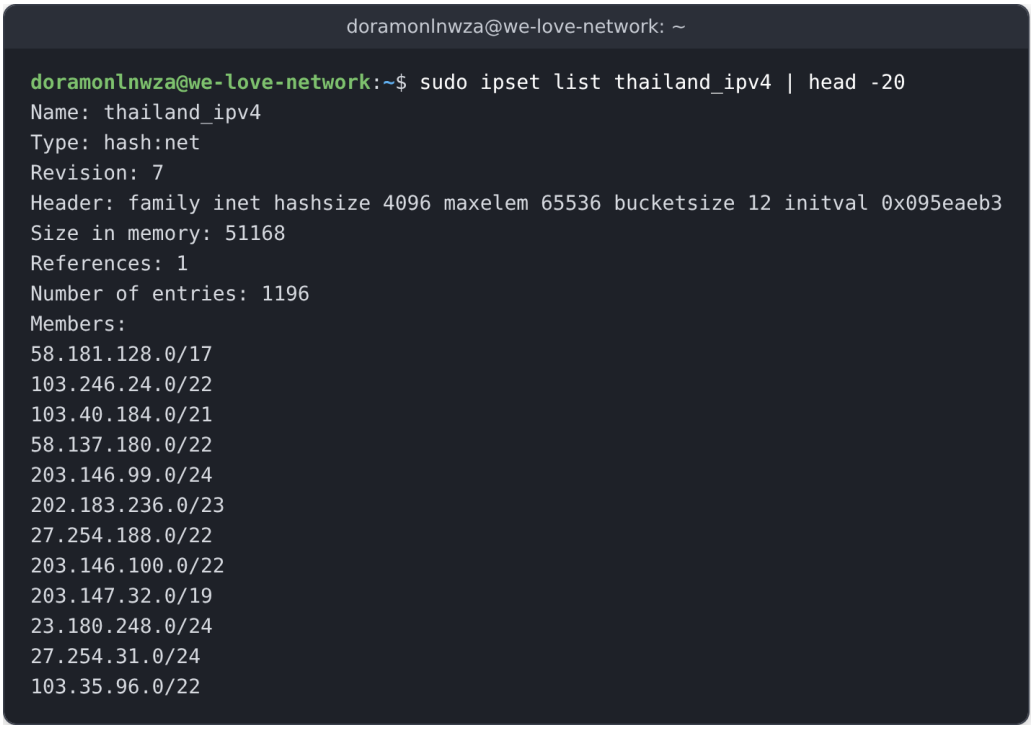
พอร์ต / บริการ	นโยบาย	เหตุผล
TCP 80 (HTTP)	เปิดใน GCP; iptables รับเฉพาะ IP ไทย	redirect ไป HTTPS และใช้ตอนออกใบรับรอง
TCP 443 (HTTPS)	เปิดใน GCP; iptables รับเฉพาะ IP ไทย และ Cloudflare (บทที่ 5)	ช่องทางหลักของเว็บไซต์
TCP 22 (SSH)	เปิด; ไม่กรองตามประเทศ; ใช้กุญแจเท่านั้น	ให้ผู้ดูแลเข้าถึงได้เสมอ ป้องกันการล็อกตัวเองออก
TCP 3306 (MySQL)	ไม่เปิดสู่ภายนอก	WordPress เชื่อมต่อผ่าน localhost
TCP 8765 (Dashboard)	ฟังเฉพาะ 127.0.0.1	เข้าถึงผ่าน Nginx Reverse Proxy เท่านั้น
PHP-FPM	Unix socket ไม่มีพอร์ตเครือข่าย	ไม่มีช่องทางเข้าจากภายนอก
TCP 20201 / 20202	ไม่เปิด	ไม่มีบริการที่ใช้งาน

3.2 การเตรียมรายการ IP ประเทศไทย

1. ดาวน์โหลดรายการ IPv4 CIDR ของประเทศไทยจาก IPdeny (`th.zone`) บันทึกเป็นไฟล์ `block_IP.txt`

2. ตรวจรูปแบบ CIDR ทุกบรรทัดด้วยสคริปต์ Python พบรายการถูกต้อง 1,196 รายการ และผิดรูปแบบ 0 รายการ
3. นำเข้า ipset ชื่อ **thailand_ipv4** ชนิด **hash:net** ซึ่งเก็บเป็นช่วงเครือข่ายและค้นหาด้วยตารางแฮชได้เร็ว ไม่ว่ารายการจะยาวเท่าใด

```
sudo ipset create thailand_ipv4 hash:net family inet hashsize 4096 maxelem 65536
while read cidr; do sudo ipset add thailand_ipv4 "$cidr"; done < block_IP.txt
```

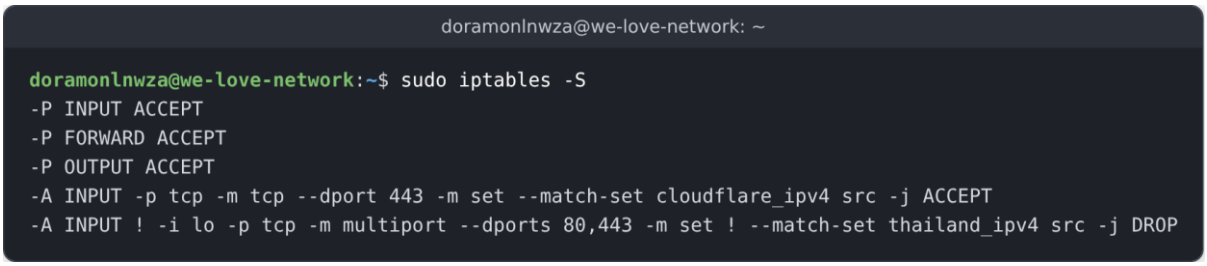


ภาพที่ 3.1 ผลคำสั่ง ipset list thailand_ipv4 แสดงรายการ IP ประเทศไทย 1,196 ช่วง

ตารางที่ 3.2 ความหมายของข้อมูลใน ipset

ข้อมูล	ความหมาย
Type: hash:net	เก็บเป็นช่วงเครือข่าย (CIDR) ไม่ใช่ IP รายตัว
family inet	เก็บที่อยู่ IPv4
hashsize 4096 / maxelem 65536	ขนาดตารางแฮชเริ่มต้น และจำนวนรายการสูงสุดที่รองรับ
Size in memory: 51168	ใช้หน่วยความจำประมาณ 51 KB
References: 1	มีกฎ iptables อ้างอิงชุดนี้อยู่ 1 กฎ
Number of entries: 1196	จำนวนช่วง IP ของประเทศไทยที่บันทึกไว้

3.3 กฎ iptables



ภาพที่ 3.2 ผลคำสั่ง iptables -S แสดงนโยบายเริ่มต้นและกฎกรองตามประเทศ

ผลในภาพที่ 3.2 มีความหมายดังนี้ นโยบายเริ่มต้นของ INPUT, FORWARD และ OUTPUT เป็น ACCEPT แต่กฎที่เพิ่มเข้าไปจะถูกตรวจตามลำดับจากบนลงล่าง และหยุดที่กฎแรกที่ตรง

```
-A INPUT -p tcp -m tcp --dport 443 -m set --match-set cloudflare_ipv4 src -j ACCEPT
-A INPUT ! -i lo -p tcp -m multiport --dports 80,443 -m set ! --match-set thailand_ipv4 src
-j DROP
```

- **กฎที่ 1** รับการเชื่อมต่อ TCP พอร์ต 443 จาก IP ในชุด `cloudflare_ipv4` เพิ่มใน Stage 2 เพื่อให้ Cloudflare Worker ดึงไฟล์จากเซิร์ฟเวอร์ได้ (อธิบายในบทที่ 5)
- **กฎที่ 2** ทิ้ง (DROP) แพ็กเก็ตที่เข้าพอร์ต 80 และ 443 จาก IP ที่ไม่อยู่ใน `thailand_ipv4` โดยยกเว้น loopback (`! -i lo`) เพื่อไม่ให้กระทบบริการภายในเครื่อง
- **ไม่มีกฎกรองพอร์ต 22** โดยเจตนา เพื่อให้ผู้ดูแลเข้าถึงเครื่องได้แม้อยู่นอกประเทศหรือรายการ IP ผิดพลาด ความปลอดภัยของ SSH อาศัยการยืนยันด้วยกุญแจแทน

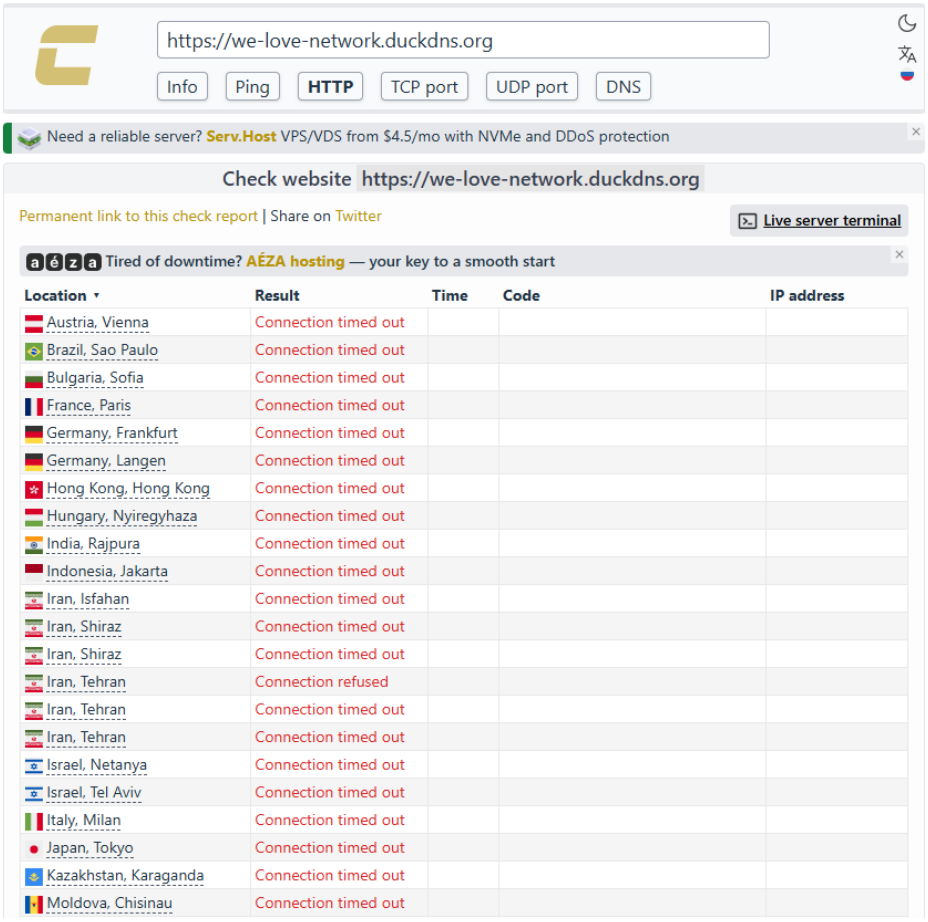
การใช้ DROP แทน REJECT ทำให้เซิร์ฟเวอร์ไม่ตอบกลับใด ๆ ผู้เชื่อมต่อจากต่างประเทศจึงรอจนหมดเวลา (timeout) และไม่รู้ด้วยซ้ำว่ามีบริการอยู่ ซึ่งยังลดภาระเซิร์ฟเวอร์จากการสแกนพอร์ตด้วย

3.4 การบันทึกกฎให้คงอยู่หลังรีบูต

- บันทึกกฎ IPv4 ไว้ที่ `/etc/iptables/rules.v4`
- บันทึกรายการ ipset ไว้ที่ `/etc/iptables/ipsets`
- เปิดใช้ `netfilter-persistent` ให้โหลดกฎอัตโนมัติเมื่อเปิดเครื่อง และตรวจไวยากรณ์กฎ IPv4 และ IPv6 ผ่านแล้ว

3.5 ผลการทดสอบ

ทดสอบจากต่างประเทศด้วยบริการ check-host.net ซึ่งส่งคำขอ HTTP ไปยังเว็บไซต์จากจุดตรวจทั่วโลกพร้อมกัน ผลคือจุดตรวจทั้ง 47 จุดใน 30 ประเทศและเขตปกครองเชื่อมต่อไม่ได้ทั้งหมด (ภาพที่ 3.3 และ 3.4)
รวมถึงประเทศเพื่อนบ้าน เช่น สิงคโปร์ เวียดนาม อินโดนีเซีย และฮ่องกง
ส่วนการเข้าจากเครือข่ายในประเทศไทยใช้งานได้ตามปกติ (ภาพที่ 4.1–4.3)



Location	Result	Time	Code	IP address
Austria, Vienna	Connection timed out			
Brazil, Sao Paulo	Connection timed out			
Bulgaria, Sofia	Connection timed out			
France, Paris	Connection timed out			
Germany, Frankfurt	Connection timed out			
Germany, Langen	Connection timed out			
Hong Kong, Hong Kong	Connection timed out			
Hungary, Nyiregyhaza	Connection timed out			
India, Rajpura	Connection timed out			
Indonesia, Jakarta	Connection timed out			
Iran, Isfahan	Connection timed out			
Iran, Shiraz	Connection timed out			
Iran, Shiraz	Connection timed out			
Iran, Tehran	Connection refused			
Iran, Tehran	Connection timed out			
Iran, Tehran	Connection timed out			
Israel, Netanya	Connection timed out			
Israel, Tel Aviv	Connection timed out			
Italy, Milan	Connection timed out			
Japan, Tokyo	Connection timed out			
Kazakhstan, Karaganda	Connection timed out			
Moldova, Chisinau	Connection timed out			

ภาพที่ 3.3 ผลทดสอบจาก check-host.net (ส่วนที่ 1): จุดตรวจในต่างประเทศได้ผล Connection timed out

Netherlands, Amsterdam	Connection timed out			
Netherlands, Meppel	Connection timed out			
Poland, Warsaw	Connection timed out			
Portugal, Viana	Connection timed out			
Russia, Moscow	Connection timed out			
Russia, Moscow	Connection timed out			
Russia, Saint Petersburg	Connection timed out			
Serbia, Belgrade	Connection timed out			
Singapore, Singapore	Connection timed out			
Slovenia, Maribor	Connection timed out			
Spain, Barcelona	Connection timed out			
Spain, Madrid	Connection timed out			
Sweden, Stockholm	Connection timed out			
Sweden, Stockholm	Connection timed out			
Switzerland, Zurich	Connection timed out			
Switzerland, Zurich	Connection timed out			
Turkey, Istanbul	Connection timed out			
Turkey, Istanbul	Connection timed out			
UK, London	Connection timed out			
Ukraine, Khmelnytskyi	Connection timed out			
Ukraine, Kyiv	Connection timed out			34.87.170.1
USA, Atlanta	Connection timed out			
USA, Los Angeles	Connection timed out			
USA, New York	Connection timed out			
Vietnam, Ho Chi Minh City	Connection timed out			

ภาพที่ 3.4 ผลทดสอบจาก check-host.net (ส่วนที่ 2): จุดตรวจในยุโรป อเมริกา และเอเชียได้ผล Connection timed out

ตารางที่ 3.3 สรุปผลการทดสอบไฟร์วอลล์

รายการทดสอบ	ผล	หมายเหตุ
เข้าจากเครือข่ายในประเทศไทย (คอมพิวเตอร์และมือถือ)	เข้าได้	ภาพที่ 4.1–4.3
เข้าจากต่างประเทศ 47 จุดตรวจ	เข้าไม่ได้ทุกจุด	46 จุด timed out ตรงกับพฤติกรรมของ DROP; 1 จุดในเตหะรานได้ connection refused ซึ่งน่าจะเกิดจากการกรองในเครือข่ายต้นทางเอง เพราะ DROP ไม่ส่งคำตอบกลับ
SSH พอร์ต 22	ยังเชื่อมต่อได้	ตามนโยบาย ไม่กรองตามประเทศ
โหลดกฎหลังรีบูตจริง	ยังไม่ได้ทดสอบ	ตั้งค่า netfilter-persistent แล้ว
การกรอง IPv6	ยังไม่ได้ตั้งค่า	ระบบยังไม่เปิดใช้ IPv6

ข้อจำกัดของการกรองตามประเทศ

- ฐานข้อมูล IP ตามประเทศระบุตำแหน่งได้ไม่สมบูรณ์ 100% และการจัดสรร IP เปลี่ยนได้ จึงควรอัปเดตรายการจาก IPdeny เป็นระยะ
- ผู้ใช้ต่างประเทศที่ใช้ VPN ซึ่งออกด้วย IP ไทยยังเข้าถึงได้ การกรองตามประเทศจึงเป็นเพียงชั้นหนึ่งของการป้องกัน
- หากเปิดใช้ IPv6 ในอนาคต ต้องสร้างชุด IPv6 และกฎ ip6tables เพิ่ม มิฉะนั้นผู้ใช้ IPv6 จะไม่ถูกกรอง
- ยังไม่ได้ทดสอบการโหลดกฎหลังรีบูตเครื่องจริง

บทที่ 4

ด่านที่ 3: เว็บไซต์ WordPress ประชาสัมพันธ์หลักสูตร

โจทย์ด่านที่ 3

สร้าง WordPress แนะนำหลักสูตรวิทยาการคอมพิวเตอร์เพื่อโปรโมทให้นักเรียนที่สนใจรับรู้ข่าวสาร ใช้ theme และ plugin ตามสะดวก ให้ดูสวยงาม

4.1 การออกแบบเว็บไซต์

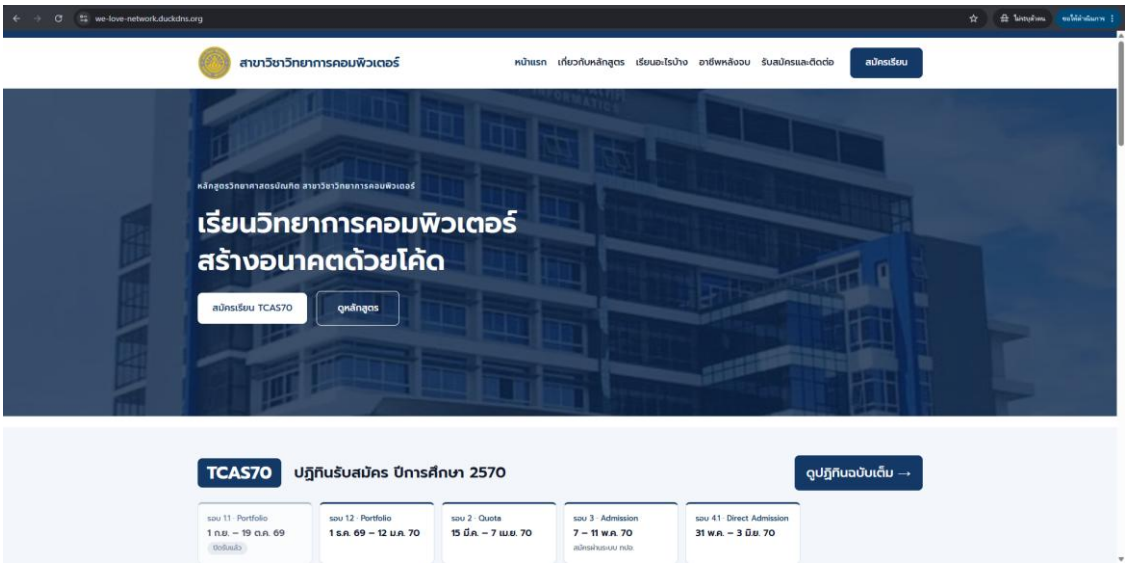
เว็บไซต์ใช้ WordPress 7.1.3 กับธีม Twenty Twenty-Five ซึ่งเป็นธีมมาตรฐานแบบ Block Theme ของ WordPress ปรับแต่งผ่าน Site Editor โดยใช้โทนสีน้ำเงินกรมท่าเข้ากับตรามหาวิทยาลัย ใช้ฟอนต์ Prompt สำหรับภาษาไทย และสร้างส่วนปฏิทินรับสมัคร TCAS70 ด้วยบล็อก Custom HTML ที่เขียนขึ้นเอง ธีมนี้รองรับ Responsive Design จึงปรับการแสดงผลตามขนาดหน้าจอโดยอัตโนมัติ

4.2 โครงสร้างเนื้อหา

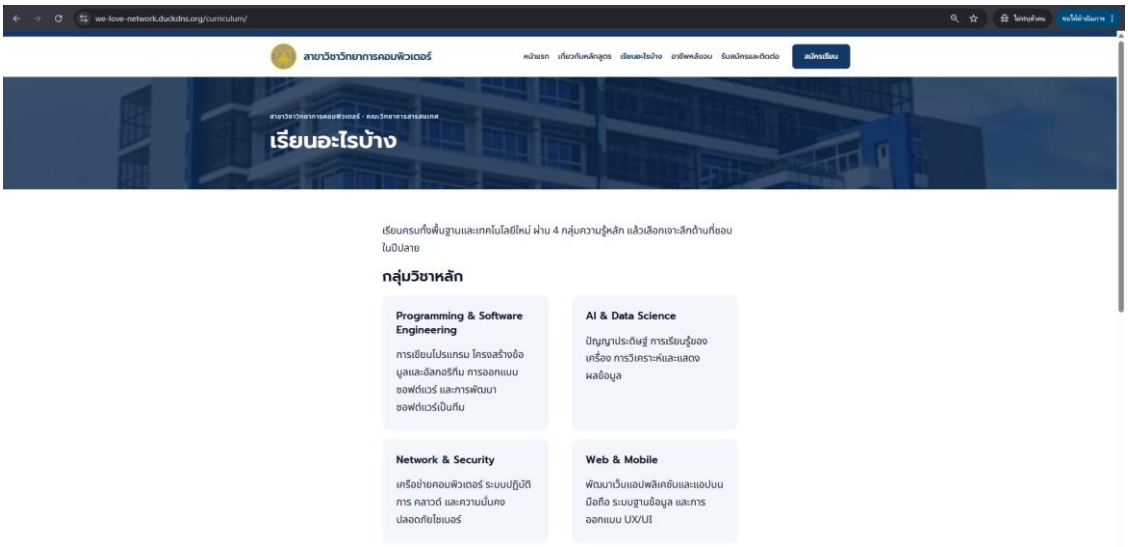
ตารางที่ 4.1 โครงสร้างหน้าเว็บไซต์

หน้า	URL	เนื้อหาหลัก
หน้าแรก	/	ข้อความหลัก "เรียนวิทยาการคอมพิวเตอร์ สร้างอนาคตด้วยโค้ด" ปุ่มสมัครเรียนและดูหลักสูตร ปฏิทินรับสมัคร TCAS70 และสายอาชีพหลังจบ
เกี่ยวกับหลักสูตร	/about/	ชื่อหลักสูตรและปริญญา ระยะเวลาเรียน ปรัชญาและวัตถุประสงค์ จุดเด่น คุณสมบัติผู้สมัคร
เรียนอะไรบ้าง	/curriculum/	กลุ่มวิชาหลัก 4 กลุ่ม (Programming & SE, AI & Data Science, Network & Security, Web & Mobile) และแผนการเรียน 4 ปี
อาชีพหลังจบ	/careers/	ตัวอย่าง 6 สายงาน และทักษะที่ได้ระหว่างเรียน
รับสมัครและติดต่อ	/apply/	ปฏิทินการรับสมัคร TCAS70 ฉบับเต็มและหมายเหตุ

4.3 ภาพหน้าเว็บไซต์



ภาพที่ 4.1 หน้าแรกของเว็บไซต์บนคอมพิวเตอร์ ผ่าน https://we-love-network.duckdns.org



ภาพที่ 4.2 หน้า "เรียนอะไรบ้าง" แสดงกลุ่มวิชาหลัก 4 กลุ่มของหลักสูตร



ภาพที่ 4.3 หน้าแรกบนสมาร์ตโฟน เมนูเปลี่ยนเป็นปุ่มแฮมเบอร์เกอร์ตามขนาดหน้าจอ

4.4 ปลั๊กอินและแนวทางความปลอดภัยของ WordPress

โจทย์อนุญาตให้ใช้ theme และ plugin ตามสะดวก กลุ่มเลือกสร้างหน้าเว็บด้วยบล็อกมาตรฐานของ WordPress และไม่เปิดใช้ปลั๊กอินเสริมจากภายนอก เพราะปลั๊กอินของผู้พัฒนารายอื่นเป็นช่องทางที่เว็บไซต์ WordPress ถูกโจมตีบ่อยที่สุด ปลั๊กอินที่ติดมากับการติดตั้ง (Akismet และ Hello Dolly) ไม่ได้เปิดใช้งาน และได้ลบ Hello Dolly ออกแล้ว ปลั๊กอินเพียงตัวเดียวที่ทำงานคือ Must-Use Plugin ที่กลุ่มเขียนขึ้นเองสำหรับ Stage 2 (หัวข้อ 5.3)

- อัปเดต WordPress ธีม และปลั๊กอินอย่างสม่ำเสมอ
- ลบส่วนขยายที่ไม่ได้ใช้งานออกจากเซิร์ฟเวอร์
- ใช้บัญชีผู้ดูแลเท่าที่จำเป็น และตั้งรหัสผ่านที่คาดเดายาก
- สำรองฐานข้อมูลและไฟล์เป็นระยะ พร้อมทดสอบการกู้คืน
- ปฏิเสธการเข้าถึงไฟล์ซ่อน เช่น .htaccess ด้วยกฎ `location ~ /\.ht { deny all; }`

บทที่ 5

Stage 2: ยกระดับความปลอดภัยและประสิทธิภาพ

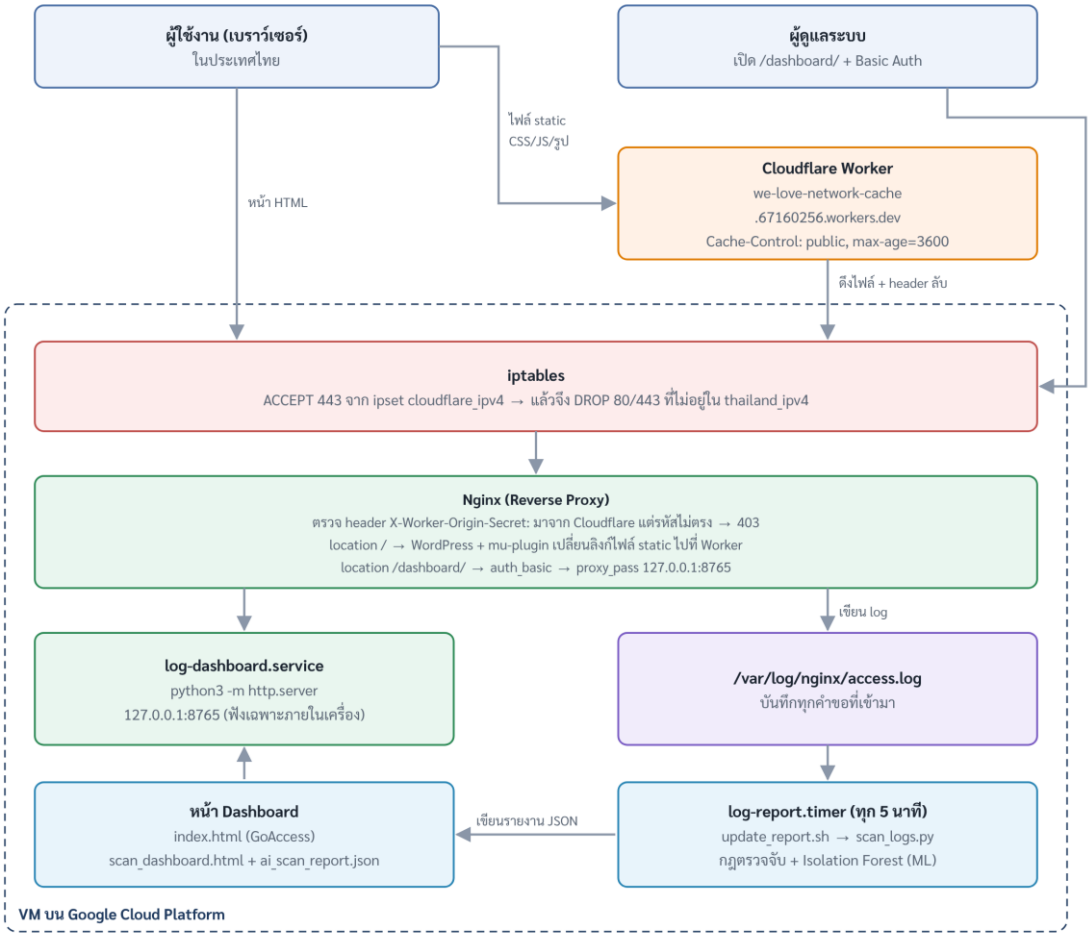
โจทย์ Stage 2

ปรับโครงสร้างของสถาปัตยกรรมที่ออกแบบไว้ให้มีความปลอดภัยสูงขึ้นหรือมีประสิทธิภาพสูงขึ้น เช่น

- ทำ Reverse Proxy
- ใช้ Cloudflare Cache
- นำ AI มา scan log ตามระยะเวลาที่กำหนด เพื่อสร้างเป็น Dashboard สำหรับตรวจสอบการโจมตีของบุคคลไม่พึงประสงค์

5.1 ภาพรวมสถาปัตยกรรม Stage 2

Stage 2 เพิ่มองค์ประกอบสามส่วนจาก Stage 1 ได้แก่ (1) Nginx ทำหน้าที่ Reverse Proxy ไปยังบริการ Dashboard ภายในเครื่อง (2) Cloudflare Worker เป็นแคชสำหรับไฟล์ static และ (3) ระบบสแกน Log ตามรอบเวลาด้วยกฎตรวจจับร่วมกับโมเดล Machine Learning แล้วแสดงผลบน Dashboard ภาพที่ 5.1 แสดงเส้นทางข้อมูลทั้งหมด



ภาพที่ 5.1 แผนภาพสถาปัตยกรรม Stage 2: Cloudflare Worker, Reverse Proxy และระบบสแกน Log

5.2 Reverse Proxy สำหรับ Dashboard

Reverse Proxy คือเซิร์ฟเวอร์ตัวกลางที่รับคำขอจากผู้ใช้แล้วส่งต่อไปยังบริการด้านหลัง ผู้ใช้เห็นเพียงเซิร์ฟเวอร์ตัวกลาง ระบบนี้ให้บริการ Dashboard ด้วย `python3 -m http.server` ที่ฟังเฉพาะ `127.0.0.1:8765` ภายนอกจึงเข้าถึงพอร์ตนี้โดยตรงไม่ได้ ต้องผ่าน Nginx ซึ่งเพิ่ม HTTPS และการยืนยันตัวตนให้ก่อนส่งต่อ

```
location = /dashboard {
    return 301 /dashboard/;
}
location /dashboard/ {
    auth_basic "Dashboard Login";
    auth_basic_user_file /etc/nginx/.dashboard_htpasswd;


    proxy_pass http://127.0.0.1:8765/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

- **auth_basic** บังคับให้กรอกชื่อผู้ใช้และรหัสผ่านก่อนเข้าดู Dashboard โดยรหัสผ่านเก็บแบบแฮชในไฟล์ `.dashboard_htpasswd` และส่งผ่าน HTTPS จึงไม่ถูกดักอ่านระหว่างทาง

- **proxy_pass** ส่งคำขอต่อไปยังบริการภายในเครื่อง ทำให้บริการด้านหลังไม่ต้องจัดการ TLS หรือการยืนยันตัวตนเอง
- **proxy_set_header** ส่ง IP จริงของผู้ใช้และโปรโตคอลเดิมไปให้บริการด้านหลัง เพื่อให้บันทึกข้อมูลได้ถูกต้อง

```
doramonlnwza@we-love-network: ~
doramonlnwza@we-love-network:~$ sudo nginx -T 2>/dev/null | grep -nE "proxy_pass|auth_basic|server_name|listen"
25:     # server_names_hash_bucket_size 64;
26:     # server_name_in_redirect off;
75:#            listen     localhost:110;
81:#            listen     localhost:143;
241:    server_name we-love-network.duckdns.org;
252:    auth_basic "Dashboard Login";
253:        auth_basic_user_file /etc/nginx/.dashboard_htpasswd;
255:        proxy_pass http://127.0.0.1:8765/;
274:    listen 443 ssl; # managed by Certbot
290:    listen 80;
291:    server_name we-love-network.duckdns.org;
336:fastcgi_param  SERVER_NAME        $server_name;
```

ภาพที่ 5.2 ค่าตั้ง Nginx ที่เกี่ยวข้อง: server_name, auth_basic, proxy_pass และ listen

บริการ Dashboard ทำงานเป็น systemd service ชื่อ `log-dashboard.service` ด้วยสิทธิ์ผู้ใช้ทั่วไป (ไม่ใช่ root) และตั้ง `Restart=always` ให้เริ่มใหม่อัตโนมัติหากหยุดทำงาน

```
[Service]
Type=simple
User=doramonlnwza
WorkingDirectory=/home/doramonlnwza/log-dashboard
ExecStart=/usr/bin/python3 -m http.server 8765 --bind 127.0.0.1
Restart=always
RestartSec=5
```

5.3 Cloudflare Worker Cache สำหรับไฟล์ static

5.3.1 ปัญหาและแนวทางที่เลือก

วิธีทั่วไปในการใช้ Cloudflare คือย้าย nameserver ของโดเมนไปที่ Cloudflare แล้วเปิด proxy แต่วิธีนี้ใช้กับโครงงานนี้ไม่ได้ด้วยเหตุผลสองข้อ ข้อแรก กลุ่มเป็นเจ้าของเพียงโดเมนย่อยของ `duckdns.org` จึงย้าย nameserver ไม่ได้ ข้อสอง หากทราฟฟิกทั้งหมดผ่าน Cloudflare เซิร์ฟเวอร์จะเห็นทุกคำขอมาจาก IP ของ Cloudflare กฎกรอง IP ไทยในบทที่ 3 จะใช้ไม่ได้ทันที

กลุ่มจึงเลือกส่งเฉพาะไฟล์ static (CSS, JavaScript, รูปภาพ และฟอนต์) ผ่าน Cloudflare Worker ที่ `we-love-network-cache.67160256.workers.dev` ส่วนหน้า HTML ยังส่งตรงจาก Nginx และผ่านไฟร์วอลล์ตามเดิม ไฟล์ static เป็นส่วนที่มีขนาดใหญ่ที่สุดของหน้าเว็บ (รูปภาพในเว็บไซต์มีขนาดถึงไฟล์ละประมาณ 5 MB) การแคชไว้ที่ Cloudflare จึงลดภาระและแบนด์วิดท์ของ VPS ได้มาก

5.3.2 Must-Use Plugin สำหรับเปลี่ยนลิงก์

กลุ่มเขียนปลั๊กอินชนิด Must-Use ชื่อ WLN Cloudflare Worker Static Assets ไว้ที่ `/var/www/wordpress/wp-content/mu-plugins/wln-worker-assets.php` ปลั๊กอินชนิดนี้ WordPress

โหลดอัตโนมัติทุกครั้งและปิดจากหน้าผู้ดูแลไม่ได้ ไฟล์เป็นของ root และมีสิทธิ์ `rw-r--r--` ทำให้ WordPress อ่านได้แต่แก้ไขไม่ได้ แม้บัญชีผู้ดูแลเว็บถูกเจาะก็ไม่สามารถแก้โค้ดส่วนนี้



ภาพที่ 5.3 Must-Use Plugin ที่กลุ่มพัฒนา แสดงในหน้า Plugins ของ WordPress

```
doramonlnwza@we-love-network: ~
doramonlnwza@we-love-network:~$ ls -la /var/www/wordpress/wp-content/mu-plugins/
total 12
drwxr-xr-x 2 root     root     4096 Oct 27 20:15 .
drwxr-xr-x 6 www-data www-data 4096 Oct 29 11:45 ..
-rw-r--r-- 1 root     root     2123 Oct 27 20:15 wln-worker-assets.php
```

ภาพที่ 5.4 ไฟล์ปลั๊กอินในโฟลเดอร์ mu-plugins เป็นของ root และแก้ไขโดย WordPress ไม่ได้

หลักการทำงานของปลั๊กอินมีดังนี้

1. ใช้ `ob_start` เก็บ HTML ทั้งหน้าไว้ก่อนส่งให้ผู้ใช้
2. ค้นหาลิงก์ที่ชี้ไปยัง `/wp-content/` หรือ `/wp-includes/` ของโดเมนตนเอง
3. เปลี่ยนเฉพาะไฟล์นามสกุล css, js, mjs, png, jpg, gif, webp, avif, svg, ico, woff, woff2, ttf, otf และ eot ให้ชี้ไปที่ Worker โดยคง path และ query string (เช่น `?ver=7.1.3`) ไว้
   เพื่อให้แคชถูกล้างเองเมื่อเวอร์ชันไฟล์เปลี่ยน
4. ไม่แตะหน้าผู้ดูแลระบบ ผู้ใช้ที่ล็อกอินอยู่ RSS feed หน้า preview และ REST API
   เพื่อไม่ให้เนื้อหาส่วนบุคคลหรือหน้าจัดการระบบไปค้างในแคช

```
add_action('template_redirect', function () {
    // Do not rewrite the WordPress admin or logged-in pages.
    if (is_admin() || is_user_logged_in() || is_feed() || is_preview()
        || (defined('REST_REQUEST') && REST_REQUEST)) {
        return;
    }
    ob_start('wln_worker_rewrite_static_assets_in_html');
}, 0);
```

5.3.3 การอนุญาต Cloudflare และการยืนยันต้นทางด้วยรหัสลับ

Worker ทำงานบนเครือข่ายของ Cloudflare ซึ่งใช้ IP ที่ไม่อยู่ในรายการ IP ไทย กลุ่มจึงสร้าง ipset `cloudflare_ipv4` จากรายการ IP ทางการของ Cloudflare และเพิ่มกฎ ACCEPT พอร์ต 443 ไว้ก่อนกฎ DROP (หัวข้อ 3.3) แต่การอนุญาต IP ของ Cloudflare ทั้งหมดเปิดช่องให้ผู้อื่นสร้าง Worker ของตัวเองแล้วยิงผ่าน Cloudflare เข้ามาเพื่อเลี่ยงการกรองประเทศได้ กลุ่มป้องกันจุดนี้โดยให้ Worker แนบ header `X-Worker-Origin-Secret` ที่มีรหัสลับทุกครั้ง และให้ Nginx ตรวจค่าดังนี้

```
map "$is_cloudflare:$http_x_worker_origin_secret" $reject_cloudflare_worker {
    default 0;
    "1:<SECRET>" 0;      # มาจาก Cloudflare และรหัสลับถูกต้อง -> ผ่าน
    ~^1: 1;              # มาจาก Cloudflare แต่ไม่มีหรือรหัสผิด -> ปฏิเสธ
}

server {
    ...
    if ($reject_cloudflare_worker) { return 403; }
}
```

ตัวแปร `$is_cloudflare` มาจากไฟล์ `/etc/nginx/cloudflare-ips.geo` ซึ่งตรวจว่า IP ต้นทางอยู่ในช่วงของ Cloudflare หรือไม่ ผลคือคำขอผ่าน Cloudflare ที่ไม่ได้มาจาก Worker ของกลุ่มจะได้รับ HTTP 403 ทันที ผู้ใช้ทั่วไปในประเทศไทยไม่ได้รับผลกระทบเพราะ `$is_cloudflare` เป็น 0 นอกจากนี้ตัว Worker เองยังตรวจประเทศของผู้ใช้อีกชั้นหนึ่ง (หัวข้อ 5.3.4) (ในรายงานนี้แทนค่ารหัสลับจริงด้วย `<SECRET>`)

5.3.4 โค้ด Cloudflare Worker

Worker เขียนด้วย JavaScript บน Cloudflare Workers โค้ดฉบับเต็มอยู่ในภาคผนวก ค ขั้นตอนการทำงานเมื่อมีคำขอเข้ามามีดังนี้

1. **ตรวจประเทศของผู้ใช้** จาก `request.cf.country` ที่ Cloudflare ระบุให้ หากไม่ใช่ `TH` ตอบ 403 ทันที ทำให้การจำกัดเฉพาะประเทศไทยตามด่านที่ 2 ครอบคลุมเส้นทางที่ผ่าน Cloudflare ด้วย
2. **ตรวจค่าตั้ง** หากยังไม่ได้ตั้งรหัสลับ `ORIGIN_SECRET` ตอบ 500 และมี path `/__health` สำหรับตรวจว่า Worker ทำงานอยู่
3. **รับเฉพาะเมธอด GET และ HEAD** เมธอดอื่นตอบ 405
4. **รับเฉพาะไฟล์ static** path ต้องขึ้นต้นด้วย `/wp-content/` หรือ `/wp-includes/` (หรือเป็น `/favicon.ico`) และต้องมีนามสกุลไฟล์ที่กำหนด มิฉะนั้นตอบ 404 Worker จึงใช้เป็นช่องทางเข้าหน้า PHP หรือหน้าผู้ดูแลไม่ได้
5. **ดึงไฟล์จากเซิร์ฟเวอร์ต้นทาง** พร้อมแนบ header `X-Worker-Origin-Secret` ที่อ่านจาก Secret ของ Cloudflare (ไม่ได้เขียนไว้ในโค้ด) และไม่ตาม redirect
6. **แคชที่ Cloudflare** ตาม status: 2xx เก็บ 3,600 วินาที, 404 เก็บ 10 วินาที, 5xx ไม่เก็บ
7. **ส่งต่อให้ผู้ใช้** กำหนด `Cache-Control: public, max-age=3600` เฉพาะเมื่อเป็น status 200 ชนิดไฟล์ static และไม่มี `Set-Cookie` กรณีอื่นตั้ง `no-store` และถ้าเซิร์ฟเวอร์ต้นทางล่มจะตอบ 502

```
// จำกัด Worker ให้ใช้จากประเทศไทย
if (request.cf?.country !== "TH") {
  return new Response("Access restricted", { status: 403, headers: { "Cache-Control": "no-
store" } });
}
...
const headers = new Headers({
  Accept: request.headers.get("Accept") || "*/*",
  "X-Worker-Origin-Secret": env.ORIGIN_SECRET,     // อ่านจาก Cloudflare Secret
});
const upstream = await fetch(target.toString(), {
  method: "GET", headers, redirect: "manual",
```

```
  cf: { cacheEverything: true,
        cacheTtlByStatus: { "200-299": CACHE_TTL, "404": 10, "500-599": 0 } },
});
```

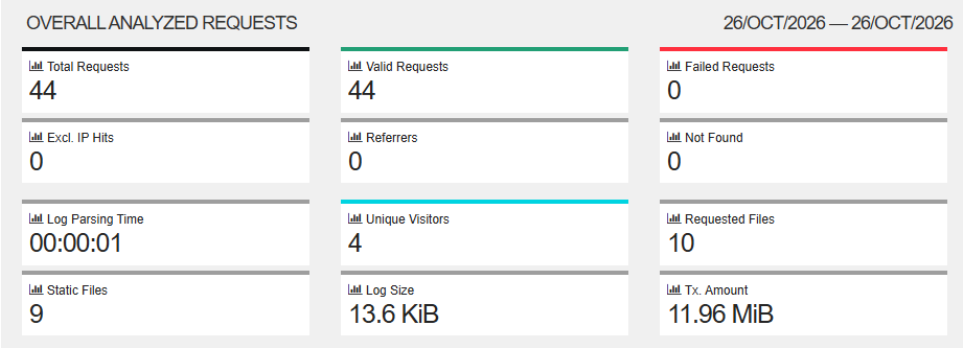
5.3.5 ผลการตรวจสอบ

ตรวจหน้าแรกของเว็บไซต์จากเบราว์เซอร์ในประเทศไทยโดยไม่ได้ล็อกอิน พบว่าทรัพยากร 11 รายการโหลดจาก `we-love-network-cache.67160256.workers.dev` และ 9 รายการโหลดจากโดเมนหลัก ไฟล์ที่ส่งผ่าน Worker มี header `Cache-Control: public, max-age=3600` หมายความว่าเบราว์เซอร์และแคชระหว่างทางเก็บไฟล์ได้ 1 ชั่วโมง นอกจากนี้ Log ของ Nginx ยังมีคำขอจาก IP กลุ่ม 172.68.x.x และ 104.22.x.x ซึ่งเป็นของ Cloudflare (ภาพที่ 5.7) ยืนยันว่า Worker ดึงไฟล์จากเซิร์ฟเวอร์ต้นทางได้จริงผ่านกฎ `cloudflare_ipv4`

5.4 ระบบสแกน Log และ Dashboard

5.4.1 GoAccess: ภาพรวมทราฟฟิก

หน้า `/dashboard/` เป็นรายงาน HTML จาก GoAccess v1.8.1 ซึ่งวิเคราะห์ `access.log` ของ Nginx และแสดงสถิติ เช่น จำนวนคำขอทั้งหมด ผู้เข้าชมที่ไม่ซ้ำ ไฟล์ที่ถูกเรียกมากที่สุด ระบบปฏิบัติการ เบราว์เซอร์ ช่วงเวลา และ HTTP status code ภาพที่ 5.5 แสดงส่วนสถิติภาพรวมของรายงานที่สร้างเมื่อวันที่ 26 ตุลาคม 2569 มีคำขอ 44 รายการ ผู้เข้าชม 4 ราย ไฟล์ที่ถูกเรียก 10 ไฟล์ (เป็นไฟล์ static 9 ไฟล์) ปริมาณข้อมูลที่ส่งออก 11.96 MiB และไม่มีคำขอที่ล้มเหลวหรือ 404



ภาพที่ 5.5 สถิติภาพรวมจาก GoAccess Dashboard (รายงาน ณ วันที่ 26 ตุลาคม 2569)

5.4.2 การสแกน Log ตามรอบเวลา

การสแกนตามรอบเวลาใช้ systemd timer แทน cron เพราะดูสถานะและประวัติการทำงานได้ด้วย `systemctl` และ `journalctl` รวมถึงรันชดเชยหลังเปิดเครื่องได้ `log-report.timer` สั่งให้ `log-report.service` ทำงานครั้งแรก 2 นาทีหลังบูต และทุก 5 นาทีหลังจากนั้น

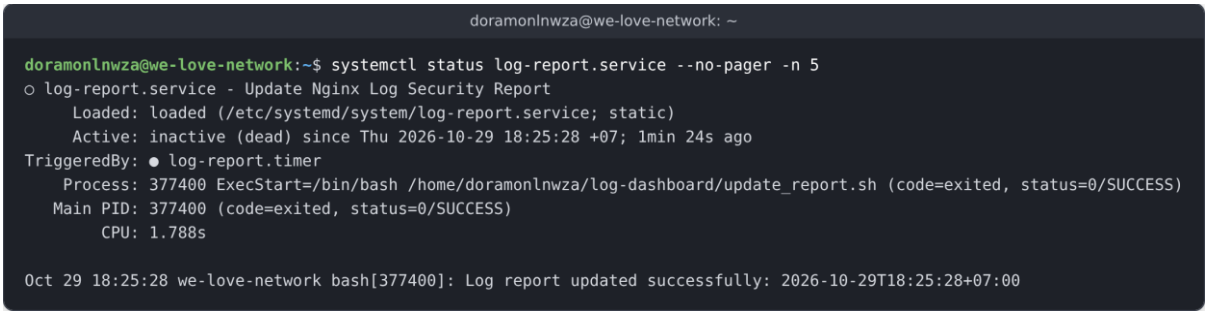
```
# /etc/systemd/system/log-report.timer
[Timer]
OnBootSec=2min
OnUnitActiveSec=5min
AccuracySec=30s
Unit=log-report.service


# /etc/systemd/system/log-report.service
```

```
[Service]
Type=oneshot
User=doramonlnwza
SupplementaryGroups=adm
ExecStart=/bin/bash /home/doramonlnwza/log-dashboard/update_report.sh
```

บริการนี้รันด้วยผู้ใช้ทั่วไปและเพิ่มเฉพาะกลุ่ม **adm** ซึ่งมีสิทธิ์อ่าน Log ของ Nginx ตามหลักสิทธิ์เท่าที่จำเป็น สคริปต์ **update_report.sh** ส่ง **access.log** ให้ **scan_logs.py** วิเคราะห์ด้วย Python ใน virtual environment **venvs/logml** ที่ติดตั้ง scikit-learn แยกจาก Python ของระบบ เขียนผลลงไฟล์ชั่วคราว ตรวจว่าเป็น JSON ที่ถูกต้องก่อน แล้วจึงแทนที่ไฟล์รายงานเดิมด้วย **mv** หากสคริปต์ทำงานผิดพลาด Dashboard จะยังแสดงรายงานล่าสุดที่ถูกต้องอยู่

```
/home/doramonlnwza/venvs/logml/bin/python "$DIR/scan_logs.py" < "$LOG" > "$TMP"
/usr/bin/python3 -m json.tool "$TMP" > /dev/null    # ตรวจรูปแบบ JSON
chmod 644 "$TMP"
mv "$TMP" "$REPORT"                                 # แทนที่แบบ atomic
```



ภาพที่ 5.6 สถานะของ log-report.service หลังทำงานรอบล่าสุด: จบการทำงานด้วย status=0/SUCCESS

สถานะ **inactive (dead)** ในภาพที่ 5.6 เป็นพฤติกรรมปกติของ service ชนิด **oneshot** ที่ทำงานจบแล้วหยุดรอ timer เรียกรอบถัดไป ส่วนบรรทัดสุดท้ายยืนยันว่ารายงานถูกสร้างใหม่สำเร็จ

5.4.3 กฎการตรวจจับ (Rule-based)

scan_logs.py อ่าน Log รูปแบบ Nginx Combined ทีละบรรทัด ตรวจ path ทั้งแบบดิบและแบบถอดรหัส URL แล้ว (ป้องกันการหลบด้วยการเข้ารหัส เช่น **%2e%2e**) และจัดระดับความเสี่ยงตามตารางที่ 5.1

ตารางที่ 5.1 กฎการตรวจจับใน scan_logs.py

ประเภท	รูปแบบที่ตรวจ	ระดับ
SQL injection pattern	union … select, or 1=1, sleep(, benchmark(	high
Path traversal	../ หรือ %2e%2e	high
Sensitive file scan	/.env, /wp-config.php, /.git, /etc/passwd, /phpmyadmin	medium
Common exploit scan	wp-login.php, xmlrpc.php, /cgi-bin/, phpunit	medium
ปริมาณคำขอสูง	IP เดียวมีคำขอ 30 ครั้งขึ้นไป (ไม่นับการรีเฟรช Dashboard)	review

การไม่นับคำขอของ Dashboard เป็นผลจากการแก้ปัญหาระหว่างพัฒนา เดิมหน้า Dashboard ที่โหลดไฟล์รายงานซ้ำทำให้ IP ของผู้ดูแลถูกตั้งธงว่าส่งคำขอมากผิดปกติ กลุ่มจึงแก้สคริปต์ให้ตัด path ของ Dashboard ออกก่อนนับ (ไฟล์สำรอง `scan_logs.py.pre_rate_fix` คือเวอร์ชันก่อนแก้)

5.4.4 การตรวจจับความผิดปกติด้วย Machine Learning (Isolation Forest)

กฎตรวจจับจับได้เฉพาะรูปแบบที่รู้ล่วงหน้า กลุ่มจึงเพิ่มโมเดล Machine Learning ชื่อ Isolation Forest จากไลบรารี scikit-learn เพื่อหาผู้ใช้ที่มีพฤติกรรมต่างจากผู้ใช้ส่วนใหญ่ แม้ไม่ตรงกฎข้อใดเลย Isolation Forest เป็นการเรียนรู้แบบไม่มีผู้สอน (unsupervised learning) จึงไม่ต้องมีข้อมูลที่ติดป้ายว่าเป็นการโจมตีมาก่อน หลักการคือสร้างต้นไม้ตัดแบ่งข้อมูลแบบสุ่มจำนวนมาก ข้อมูลที่ผิดปกติจะถูกแยกออกได้ด้วยการตัดเพียงไม่กี่ครั้ง จึงได้คะแนนความผิดปกติ (anomaly score) สูง

ทุกรอบการสแกน สคริปต์สรุปพฤติกรรมของแต่ละ IP เป็นคุณลักษณะ (feature) 5 ค่า ตามตารางที่ 5.2 โดยไม่นับคำขอของ Dashboard และคำขอจากตัวเซิร์ฟเวอร์เอง (`127.0.0.1`)

ตารางที่ 5.2 คุณลักษณะที่ใช้ฝึกโมเดล Isolation Forest

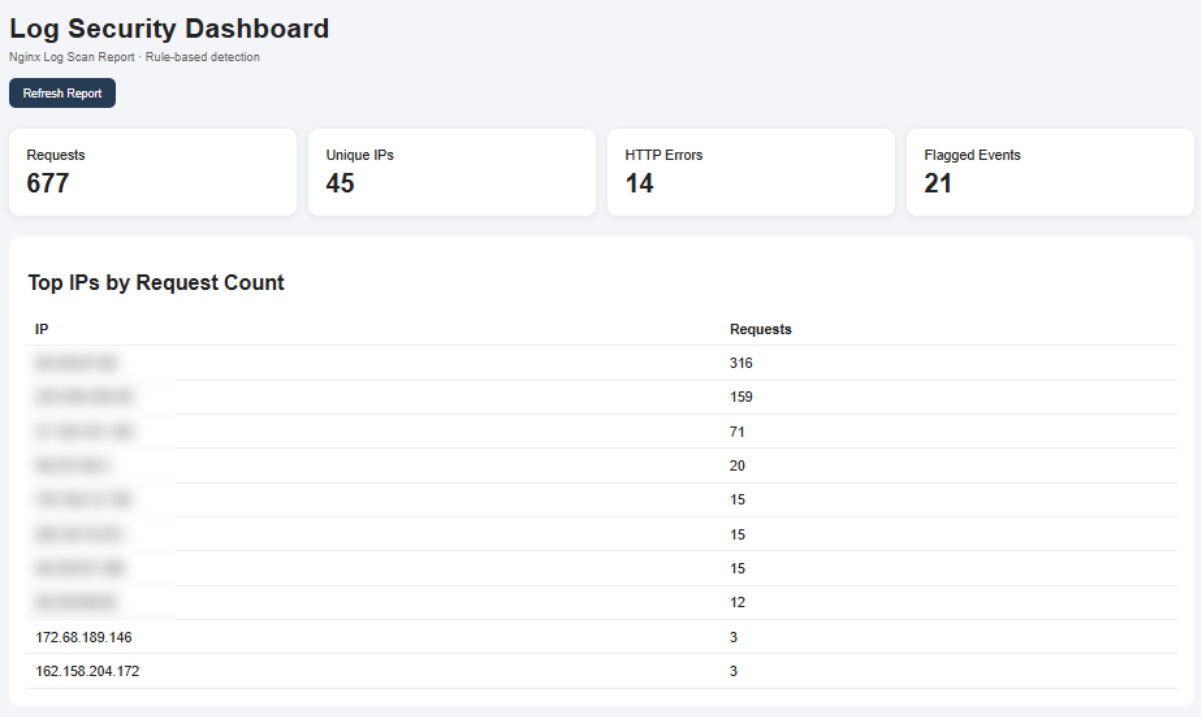
คุณลักษณะ	วิธีคำนวณ	พฤติกรรมที่สะท้อน
ปริมาณคำขอ	log(1 + จำนวนคำขอ)	การยิงคำขอจำนวนมาก (ใช้ log เพื่อลดผลของค่าที่สูงมาก)
สัดส่วน error	คำขอที่ได้ status ≥ 400 / คำขอทั้งหมด	การสุ่มหา path ที่ไม่มีอยู่จริง
สัดส่วน path ไม่ซ้ำ	จำนวน path ที่ต่างกัน / คำขอทั้งหมด	การไล่สแกนหลาย URL
สัดส่วน POST	คำขอ POST / คำขอทั้งหมด	การส่งฟอร์มหรือเดารหัสผ่านซ้ำ
สัดส่วนที่ตรงกฎ	คำขอที่ตรงกฎตรวจจับ / คำขอทั้งหมด	ความเกี่ยวข้องกับรูปแบบการโจมตีที่รู้จัก

```
model = IsolationForest(n_estimators=200, contamination="auto", random_state=42)
model.fit(rows)                          # rows = ค่าคุณลักษณะ 5 ค่าของแต่ละ IP
labels = model.predict(rows)             # -1 = ผิดปกติ, 1 = ปกติ
scores = model.score_samples(rows)       # ยิ่งติดลบมาก ยิ่งผิดปกติ
```

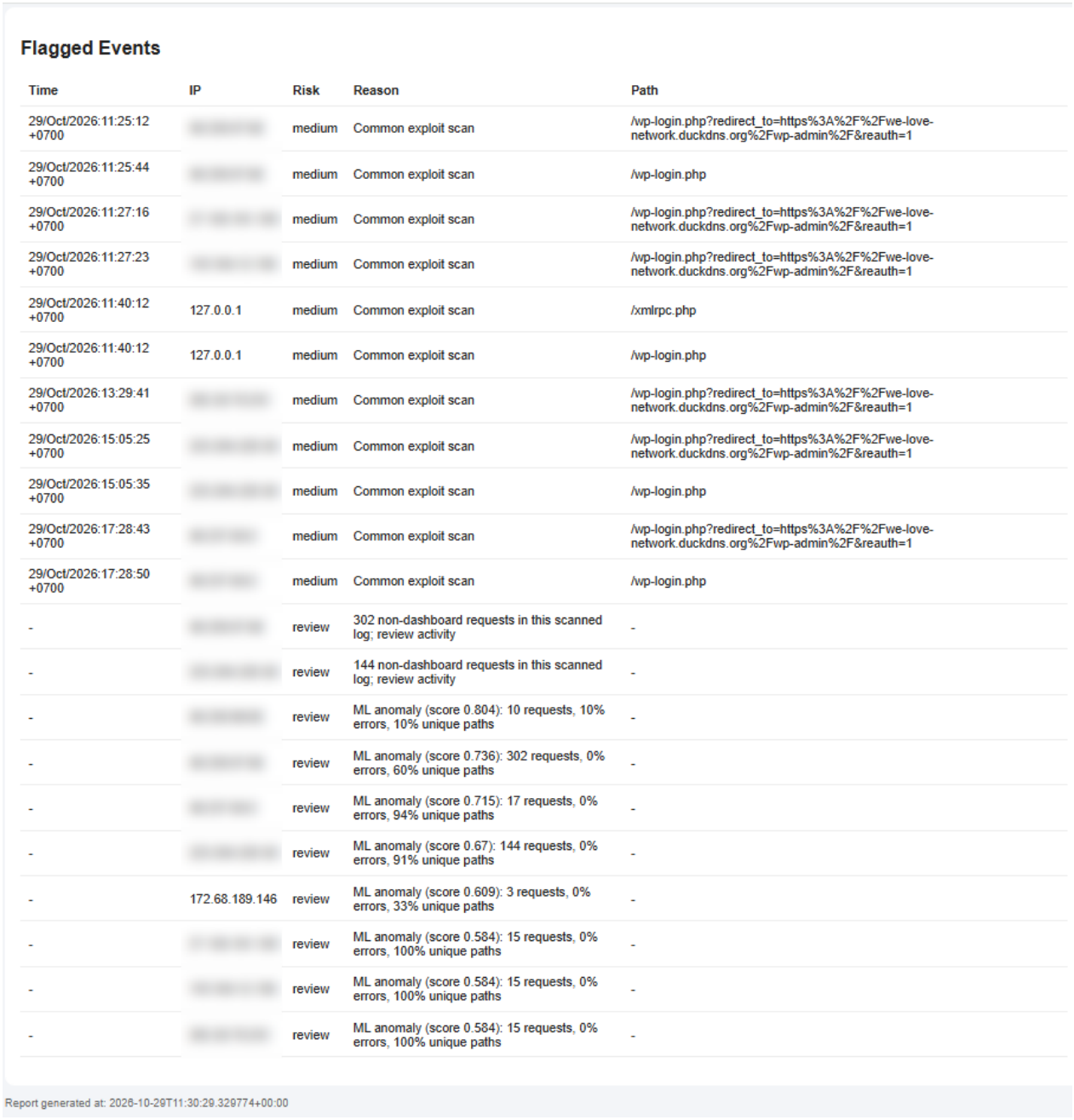
- **n_estimators=200** ใช้ต้นไม้ 200 ต้นเพื่อให้คะแนนเสถียร และ **random_state=42** ทำให้ผลซ้ำได้ทุกครั้งที่รัน
- **contamination="auto"** ให้โมเดลกำหนดเกณฑ์แบ่งปกติกับผิดปกติเองตามวิธีของงานวิจัยต้นฉบับ
- **ต้องมีอย่างน้อย 20 IP** โมเดลจึงจะทำงาน หากข้อมูลน้อยกว่านั้นผลจะไม่น่าเชื่อถือ ระบบจะใช้เฉพาะกฎตรวจจับและระบุสถานะว่า skipped
- หากไม่มี scikit-learn หรือโมเดลเกิดข้อผิดพลาด สคริปต์ยังสร้างรายงานจากกฎตรวจจับได้ตามปกติ Dashboard จึงไม่ล่มเพราะส่วน ML
- IP ที่โมเดลจัดว่าผิดปกติจะถูกเพิ่มเป็นเหตุการณ์ระดับ review พร้อมคะแนนและค่าคุณลักษณะ ส่วนรายงาน JSON ระบุ `scan_type` เป็น `rules + Isolation Forest` เมื่อโมเดลทำงานสำเร็จ

5.4.5 Log Security Dashboard และผลการวิเคราะห์

หน้า `/dashboard/scan_dashboard.html` อ่าน `ai_scan_report.json` แล้วแสดงตัวเลขสรุป 4 ค่า ตาราง IP ที่มีคำขอมากที่สุด และตารางเหตุการณ์ที่ถูกตั้งธงทั้งจากกฎตรวจจับและจากโมเดล ข้อมูลจาก Log ถูกใส่ลงหน้าเว็บด้วย `textContent` ไม่ใช้ `innerHTML` ดังนั้นแม้ผู้โจมตีจะฝังโค้ด JavaScript มาใน URL ที่ถูกบันทึกใน Log โค้ดนั้นก็จะไม่ถูกรันบนหน้า Dashboard (ป้องกัน XSS)



ภาพที่ 5.7 Log Security Dashboard: ตัวเลขสรุปและ IP ที่มีคำขอมากที่สุด (ปิดบัง IP ของผู้ใช้)



Flagged Events

Time	IP	Risk	Reason	Path
29/Oct/2026:11:25:12 +0700	[redacted]	medium	Common exploit scan	/wp-login.php?redirect_to=https%3A%2F%2Fwe-love-network.duckdns.org%2Fwp-admin%2F&reauth=1
29/Oct/2026:11:25:44 +0700	[redacted]	medium	Common exploit scan	/wp-login.php
29/Oct/2026:11:27:16 +0700	[redacted]	medium	Common exploit scan	/wp-login.php?redirect_to=https%3A%2F%2Fwe-love-network.duckdns.org%2Fwp-admin%2F&reauth=1
29/Oct/2026:11:27:23 +0700	[redacted]	medium	Common exploit scan	/wp-login.php?redirect_to=https%3A%2F%2Fwe-love-network.duckdns.org%2Fwp-admin%2F&reauth=1
29/Oct/2026:11:40:12 +0700	127.0.0.1	medium	Common exploit scan	/xmlrpc.php
29/Oct/2026:11:40:12 +0700	127.0.0.1	medium	Common exploit scan	/wp-login.php
29/Oct/2026:13:29:41 +0700	[redacted]	medium	Common exploit scan	/wp-login.php?redirect_to=https%3A%2F%2Fwe-love-network.duckdns.org%2Fwp-admin%2F&reauth=1
29/Oct/2026:15:05:25 +0700	[redacted]	medium	Common exploit scan	/wp-login.php?redirect_to=https%3A%2F%2Fwe-love-network.duckdns.org%2Fwp-admin%2F&reauth=1
29/Oct/2026:15:05:35 +0700	[redacted]	medium	Common exploit scan	/wp-login.php
29/Oct/2026:17:28:43 +0700	[redacted]	medium	Common exploit scan	/wp-login.php?redirect_to=https%3A%2F%2Fwe-love-network.duckdns.org%2Fwp-admin%2F&reauth=1
29/Oct/2026:17:28:50 +0700	[redacted]	medium	Common exploit scan	/wp-login.php
-	[redacted]	review	302 non-dashboard requests in this scanned log; review activity	-
-	[redacted]	review	144 non-dashboard requests in this scanned log; review activity	-
-	[redacted]	review	ML anomaly (score 0.804): 10 requests, 10% errors, 10% unique paths	-
-	[redacted]	review	ML anomaly (score 0.736): 302 requests, 0% errors, 60% unique paths	-
-	[redacted]	review	ML anomaly (score 0.715): 17 requests, 0% errors, 94% unique paths	-
-	[redacted]	review	ML anomaly (score 0.67): 144 requests, 0% errors, 91% unique paths	-
-	172.68.189.146	review	ML anomaly (score 0.609): 3 requests, 0% errors, 33% unique paths	-
-	[redacted]	review	ML anomaly (score 0.584): 15 requests, 0% errors, 100% unique paths	-
-	[redacted]	review	ML anomaly (score 0.584): 15 requests, 0% errors, 100% unique paths	-
-	[redacted]	review	ML anomaly (score 0.584): 15 requests, 0% errors, 100% unique paths	-

Report generated at: 2026-10-29T11:30:29.329774+00:00

ภาพที่ 5.8 เหตุการณ์ที่ถูกตั้งธงจากกฎตรวจจับและจากโมเดล Isolation Forest (ปิดบัง IP ของผู้ใช้)

รายงาน ณ วันที่ 29 ตุลาคม 2569 เวลา 18:30 น. มีคำขอ 677 รายการจาก 45 IP เกิด HTTP error 14 ครั้ง และมีเหตุการณ์ถูกตั้งธง 21 รายการ สรุปตามที่มาได้ดังตารางที่ 5.3

ตารางที่ 5.3 สรุปเหตุการณ์ที่ถูกตั้งธง ณ วันที่ 29 ตุลาคม 2569 18:30 น.

ที่มา	จำนวน	รายละเอียดและการตีความ
กฎ Common exploit scan	11	เข้าหน้า wp-login.php จาก IP ในไทย 6 IP และการทดสอบ wp-login.php กับ xmlrpc.php จากตัวเซิร์ฟเวอร์ (127.0.0.1) ไม่พบการเดารหัสผ่านซ้ำ

ที่มา	จำนวน	รายละเอียดและการตีความ
กฎปริมาณคำขอสูง	2	IP ในไทย 2 รายที่มีคำขอ 302 และ 144 ครั้ง รายแรกเป็นสมาชิกที่ทำงานในหน้าผู้ดูแล
โมเดล Isolation Forest	8	IP ที่พฤติกรรมต่างจากกลุ่ม จาก 44 IP ที่วิเคราะห์ (รายละเอียดในตารางที่ 5.4)
กฎ SQL injection / Path traversal / Sensitive file	0	ไม่พบ

ตารางที่ 5.4 IP ที่โมเดล Isolation Forest จัดว่าผิดปกติ

IP	คะแนน	คำขอ	error	path ไม่ซ้ำ	POST	ข้อสังเกต
49.230.x.x	0.804	10	10%	10%	100%	ส่งแต่ POST เกือบทั้งหมด
49.230.x.x	0.736	302	0%	60%	22%	สมาชิกทำงานใน wp-admin
49.237.x.x	0.715	17	0%	94%	6%	เปิดหลายหน้าในเวลาสั้น
223.204.x.x	0.670	144	0%	91%	3%	เปิดหลายหน้า คำขอสูง
172.68.x.x	0.609	3	0%	33%	0%	IP ของ Cloudflare Worker
27.130.x.x	0.584	15	0%	100%	0%	เปิดทุกหน้าไม่ซ้ำ
110.164.x.x	0.584	15	0%	100%	0%	เปิดทุกหน้าไม่ซ้ำ
202.28.x.x	0.584	15	0%	100%	0%	เปิดทุกหน้าไม่ซ้ำ

ไม่พบรูปแบบ SQL injection, path traversal หรือการสแกนหาไฟล์สำคัญ ซึ่งสอดคล้องกับการที่ไฟร์วอลล์ทิ้งการเชื่อมต่อจากต่างประเทศตั้งแต่ชั้นเครือข่าย ทราฟฟิกที่เหลือจึงเป็นผู้ใช้ในประเทศไทยเกือบทั้งหมด IP ที่โมเดลจัดว่าผิดปกติทั้ง 8 รายมีสัดส่วน error ต่ำและแทบไม่ตรงกฎการโจมตี เมื่อตรวจซ้ำแล้วจึงประเมินว่าเป็นผู้ใช้ปกติที่มีพฤติกรรมต่างจากคนอื่น เช่น ผู้ดูแลที่ส่งคำขอจำนวนมาก ผู้ที่ส่งแต่คำขอ POST หรือ Cloudflare Worker ที่ดึงไฟล์เพียงไม่กี่ครั้ง

ข้อจำกัดของการตรวจจับด้วย Isolation Forest

- Isolation Forest บอกได้เพียงว่าพฤติกรรม "ต่างจากผู้ใช้ส่วนใหญ่" ไม่ได้ยืนยันว่าเป็นการโจมตี ผลทุกรายการจึงจัดเป็นระดับ review ที่ต้องให้คนตรวจซ้ำ
- โมเดลถูกฝึกใหม่จาก Log ชุดเดียวกับที่วิเคราะห์ทุกรอบ เมื่อทราฟฟิกส่วนใหญ่เป็นผู้ใช้ปกติ โมเดลจะยังหยิบผู้ใช้ปกติบางรายออกมาเป็นผิดปกติเสมอ (false positive)
- จำนวน IP ยังน้อย (44 IP) คะแนนจึงอาจเปลี่ยนมากเมื่อมีผู้ใช้เพิ่ม แนวทางปรับปรุงคือสะสม Log ระยะยาวเพื่อเป็นข้อมูลฝึก และกรอง IP ของผู้ดูแลและ Cloudflare ออกจากการวิเคราะห์

บทที่ 6

ปัญหา อุปสรรค และแนวทางแก้ไข

ตารางที่ 6.1 ปัญหาที่พบและวิธีแก้ไข

ปัญหา	สาเหตุ	วิธีแก้ไข
ใส่รายการ IP ไทยใน GCP Firewall ไม่ได้	รายการมี 1,196 CIDR เกินจำนวน source range ต่อกฎของ GCP	ใช้ ipset + iptables กรองภายใน VM และใช้ GCP Firewall เปิดเฉพาะพอร์ต
ใช้ Cloudflare กับโดเมนแบบปกติไม่ได้	โดเมนเป็นโดเมนย่อยของ duckdns.org จึงย้าย nameserver ไม่ได้ และ proxy ทั้งหมดจะทำให้กรอง IP ไทยไม่ได้	ใช้ Cloudflare Worker เฉพาะไฟล์ static ผ่าน mu-plugin
Worker ดึงไฟล์จากเซิร์ฟเวอร์ไม่ได้	IP ของ Cloudflare ไม่อยู่ในรายการ IP ไทย จึงโดน DROP	สร้าง ipset cloudflare_ipv4 และกฎ ACCEPT พอร์ต 443 ก่อนกฎ DROP
ผู้อื่นอาจเลี่ยงการกรองประเทศผ่าน Cloudflare	กฎอนุญาต IP ของ Cloudflare ทั้งหมด	ให้ Worker ส่ง header รหัสลับ และให้ Nginx ตอบ 403 เมื่อรหัสไม่ตรง
Dashboard แจ้งเตือน IP ผู้ดูแลผิดพลาด	การรีเฟรช Dashboard ถูกนับเป็นคำขอปริมาณสูง	ตัด path ของ Dashboard ออกก่อนนับ (rate fix)
เสี่ยงล็อกตัวเองออกจากเซิร์ฟเวอร์ขณะปิด password	หากกุญแจตั้งค่าผิดจะเข้า SSH ไม่ได้อีก	เปิด session เดิมค้างไว้และทดสอบ session ใหม่ก่อนปิด
ส่วน AI เดิมเป็นเพียงกฎตรวจจับ	scan_logs.py รุ่นแรกตรวจจับด้วย regex อย่างเดียว ยังไม่ตรงโจทย์ที่ให้ใช้ AI	เพิ่มโมเดล Isolation Forest ที่วิเคราะห์พฤติกรรมราย IP
ติดตั้ง scikit-learn ไม่ควรปนกับระบบ	Ubuntu 24.04 ไม่แนะนำให้ติดตั้งแพ็กเกจ Python ทับของระบบ	สร้าง virtual environment แยก (venvs/logml) และให้ update_report.sh เรียก Python จาก venv
โมเดลไม่น่าเชื่อถือเมื่อข้อมูลน้อย	IP ที่ใช้ฝึกมีจำนวนน้อย	กำหนดขั้นต่ำ 20 IP ถ้าน้อยกว่านั้นใช้เฉพาะกฎตรวจจับ

บทที่ 7
สรุปผลและข้อเสนอแนะ

7.1 สรุปผลการดำเนินงาน

กลุ่มติดตั้งเว็บไซต์ WordPress ประชาสัมพันธ์หลักสูตรวิทยาการคอมพิวเตอร์บน VPS ของ Google Cloud Platform ได้ครบตามด่านที่ 1–3 ของ Stage 1 ระบบให้บริการผ่าน HTTPS บนโดเมน DuckDNS ผู้ดูแลเข้าถึงเซิร์ฟเวอร์ด้วย SSH Key เท่านั้น และเว็บไซต์รับการเชื่อมต่อเฉพาะจาก IP ในประเทศไทย ซึ่งทดสอบแล้วจาก 47 จุดตรวจในต่างประเทศ ใน Stage 2 กลุ่มเพิ่ม Reverse Proxy ที่มีการยืนยันตัวตนสำหรับ Dashboard ใช้ Cloudflare Worker แคชไฟล์ static โดยไม่ทำลายการกรองประเทศ และสร้างระบบสแกน Log ทุก 5 นาที พร้อม Dashboard สองหน้า

ตารางที่ 7.1 สรุปสถานะตามข้อกำหนดของโจทย์

ข้อกำหนด	สถานะ
ด่านที่ 1: HTTPS ผ่าน DuckDNS ไปยัง WordPress บน VPS	ผ่าน
ด่านที่ 1: SSH ปิด root / ปิด password / ใช้ PKI เท่านั้น	ผ่าน
ด่านที่ 1: สถาปัตยกรรมพร้อมเวอร์ชันโปรแกรม	ผ่าน
ด่านที่ 2: firewall บล็อกการเชื่อมต่อจากต่างประเทศ	ผ่าน (IPv4)
ด่านที่ 3: เว็บไซต์ประชาสัมพันธ์หลักสูตร	ผ่าน
Stage 2: Reverse Proxy	ดำเนินการแล้ว
Stage 2: Cloudflare Cache	ดำเนินการแล้ว (Cloudflare Worker)
Stage 2: สแกน Log ตามรอบเวลาและ Dashboard	ดำเนินการแล้ว (ทุก 5 นาที)
Stage 2: ใช้ AI วิเคราะห์ Log	ดำเนินการแล้ว (Isolation Forest)

7.2 ข้อเสนอแนะ

- ทดสอบรีบูตเครื่องเพื่อยืนยันว่า netfilter-persistent โหลดกฎ iptables และ ipset กลับมาครบ
- เตรียมนโยบายกรอง IPv6 ก่อนเปิดใช้ IPv6 และตั้งงานอัปเดตรายการ IP ไทยจาก IPdeny เป็นระยะ
- จำกัด Source IP ของกฎ default-allow-ssh ใน GCP หรือใช้ Identity-Aware Proxy เพื่อลดการสแกนพอร์ต 22
- อัปเดต WordPress เป็นรุ่น 7.1.4 ที่มีให้แล้ว และพิจารณาเปลี่ยนชื่อบัญชีผู้ดูแลเว็บจาก admin ซึ่งเป็นชื่อที่ผู้โจมตีเดาก่อนเสมอ
- เปลี่ยนรหัสผ่าน Dashboard ให้คาดเดายาก และหมุนเวียนรหัสลับระหว่าง Worker กับเซิร์ฟเวอร์เป็นระยะ

- ตั้ง timer ให้ GoAccess สร้างรายงานใหม่อัตโนมัติ เช่นเดียวกับ scan_logs.py
- เพิ่มการแจ้งเตือน เช่น ส่งข้อความเมื่อพบเหตุการณ์ระดับ high และพิจารณาใช้ fail2ban บล็อก IP ที่โจมตีซ้ำ

เอกสารอ้างอิง

Cloudflare. (n.d.). Cloudflare Workers documentation. https://developers.cloudflare.com/workers/

Cloudflare. (n.d.). IP ranges. https://www.cloudflare.com/ips/

DuckDNS. (n.d.). DuckDNS free dynamic DNS. https://www.duckdns.org/

Electronic Frontier Foundation. (n.d.). Certbot documentation. https://eff-certbot.readthedocs.io/

GoAccess. (n.d.). GoAccess manual. https://goaccess.io/man

Google Cloud. (n.d.). VPC firewall rules. https://cloud.google.com/firewall/docs/firewalls

IPdeny. (n.d.). IP address blocks by country. https://www.ipdeny.com/ipblocks/

Let's Encrypt. (n.d.). Documentation. https://letsencrypt.org/docs/

Netfilter Project. (n.d.). ipset. https://ipset.netfilter.org/

NGINX. (n.d.). NGINX documentation. https://nginx.org/en/docs/

OpenBSD. (n.d.). sshd_config(5) manual page. https://man.openbsd.org/sshd_config

WordPress.org. (n.d.). Advanced administration handbook. https://developer.wordpress.org/advanced-administration/

WordPress.org. (n.d.). Must use plugins. https://developer.wordpress.org/advanced-administration/plugins/mu-plugins/

ภาคผนวก ก คำสั่งตรวจสอบระบบ

คำสั่งต่อไปนี้ใช้เก็บหลักฐานในรายงาน รันบน VPS ผ่าน SSH

```
lsb_release -a && nginx -v && php -v && mysql --version
sudo sshd -T | grep -E 'permitrootlogin|passwordauthentication|pubkeyauthentication'
sudo certbot certificates
sudo ipset list thailand_ipv4 | head -20
sudo iptables -S
sudo nginx -T 2>/dev/null | grep -nE "proxy_pass|auth_basic|server_name|listen"
sudo ss -ltnp | grep 8765
systemctl cat log-dashboard.service log-report.service log-report.timer
systemctl list-timers --all
```

ภาคผนวก ข การเพิ่มผู้ดูแลระบบรายใหม่ผ่าน SSH

เนื่องจากระบบปิดการเข้าสู่ระบบด้วยรหัสผ่านและปิด root login ผู้ดูแลรายใหม่ต้องสร้างกุญแจ Ed25519 บนเครื่องของตนเอง แล้วส่งเฉพาะ Public Key (ไฟล์ `id_ed25519.pub`) ให้ผู้ดูแลเดิมเพิ่มลงใน `/home/doramonlnwza/.ssh/authorized_keys` ห้ามส่ง Private Key ให้ผู้อื่นไม่ว่ากรณีใด

```
# ผู้ดูแลรายใหม่ (บนเครื่องของตนเอง)
ssh-keygen -t ed25519
# ส่งไฟล์ ~/.ssh/id_ed25519.pub ให้ผู้ดูแลเดิม


# หลังผู้ดูแลเดิมเพิ่ม Public Key แล้ว
ssh doramonlnwza@we-love-network.duckdns.org
```

ภาคผนวก ค โค้ด Cloudflare Worker ฉบับเต็ม

โค้ดของ Worker **we-love-network-cache** บน Cloudflare Workers รหัสลับ **ORIGIN_SECRET** ตั้งเป็น Secret ใน Cloudflare Dashboard จึงไม่ปรากฏในโค้ด

```
const ORIGIN = "https://we-love-network.duckdns.org";
const CACHE_TTL = 3600;


const STATIC_FILE =
  /\.(css|js|mjs|png|jpe?g|gif|webp|avif|svg|ico|woff2?|ttf|otf|eot)$/i;
const STATIC_CONTENT_TYPE =
  /^(text\/css|text\/javascript|application\/javascript|image\/|font\/|application\/font-
woff|application\/vnd\.ms-fontobject)/i;


export default {
  async fetch(request, env) {
    const url = new URL(request.url);


    // จำกัด Worker ให้ใช้จากประเทศไทย
    if (request.cf?.country !== "TH") {
      return new Response("Access restricted", {
        status: 403,
        headers: { "Cache-Control": "no-store" },
      });
    }


    // ตรวจสอบว่า Worker ทำงานอยู่
    if (url.pathname === "/__health") {
      return new Response("Worker is running", {
        headers: { "Cache-Control": "no-store" },
      });
    }


    if (!env.ORIGIN_SECRET) {
      return new Response("Origin secret is not configured", {
        status: 500,
        headers: { "Cache-Control": "no-store" },
      });
    }


    if (!["GET", "HEAD"].includes(request.method)) {
      return new Response("Method not allowed", {
        status: 405,
        headers: { "Cache-Control": "no-store" },
      });
    }


    // อนุญาตเฉพาะไฟล์ Static ของ WordPress
    const allowedPath =
      url.pathname.startsWith("/wp-content/") ||
      url.pathname.startsWith("/wp-includes/") ||
      url.pathname === "/favicon.ico";
```

```
    if (!allowedPath || !STATIC_FILE.test(url.pathname)) {
      return new Response("Static files only", {
        status: 404,
        headers: { "Cache-Control": "no-store" },
      });
    }


    // สร้าง URL ไปยัง VPS ต้นทาง
    const target = new URL(ORIGIN);
    target.pathname = url.pathname;
    target.search = url.search;


    const headers = new Headers({
      Accept: request.headers.get("Accept") || "*/*",
      "X-Worker-Origin-Secret": env.ORIGIN_SECRET,
    });


    try {
      const upstream = await fetch(target.toString(), {
        method: "GET",
        headers,
        redirect: "manual",
        cf: {
          cacheEverything: true,
          cacheTtlByStatus: {
            "200-299": CACHE_TTL,
            "404": 10,
            "500-599": 0,
          },
        },
      });


      const responseHeaders = new Headers(upstream.headers);
      const contentType =
        upstream.headers.get("Content-Type") || "";


      const isStaticResponse =
        upstream.status === 200 &&
        STATIC_CONTENT_TYPE.test(contentType) &&
        !upstream.headers.has("Set-Cookie");


      responseHeaders.set(
        "X-Worker-Origin-Status",
        String(upstream.status)
      );


      if (isStaticResponse) {
        responseHeaders.set(
          "Cache-Control",
          `public, max-age=${CACHE_TTL}`
        );


        responseHeaders.set(
          "Access-Control-Allow-Origin",
          "https://we-love-network.duckdns.org"
        );
      } else {
        responseHeaders.set("Cache-Control", "no-store");
      }
```

```
      return new Response(
        request.method === "HEAD" ? null : upstream.body,
        {
          status: upstream.status,
          statusText: upstream.statusText,
          headers: responseHeaders,
        }
      );
    } catch (error) {
      console.error("Origin request failed:", error);


      return new Response("Origin unavailable", {
        status: 502,
        headers: { "Cache-Control": "no-store" },
      });
    }
  },
};
```